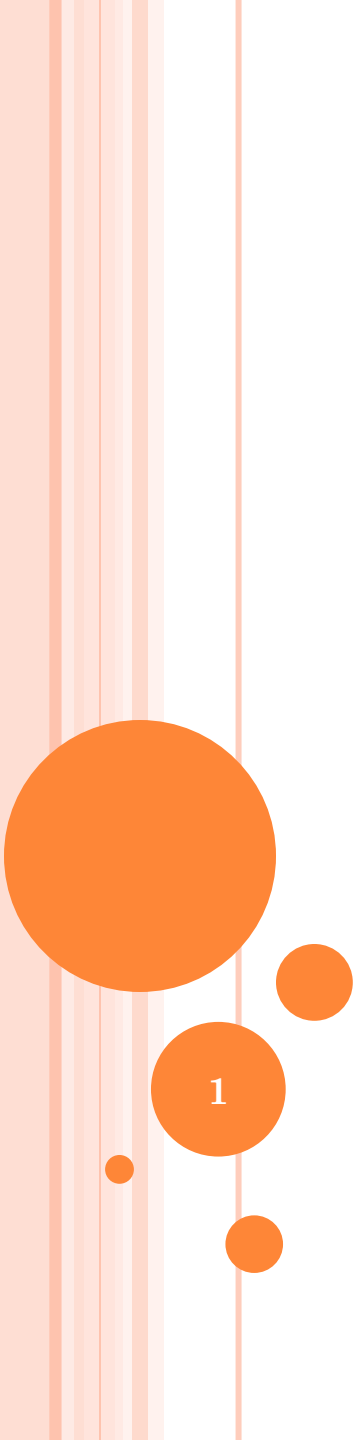




SISTEME DE OPERARE

Curs 1: Concepte introductive

Kinga Márton

**Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare**

1

CHESTIUNI ADMINISTRATIVE

○ Curs

- 14 săptămâni * 2 ore – luni: 10 - 12
- Evaluare: 5p
 - Teste scurte + Examen

○ Laborator

- 14 săptămâni * 2 ore
 - Grupa 4: marți: 16 – 20
 - Grupa 5: joi: 12 – 20
 - Grupele 1, 2, 3: vineri: 8 - 20
- Evaluare: 5p

○ Condiții de promovare

- Lab ≥ 5 pentru intrare în examen
- Nota finală ≥ 5 promovare materie

CHESTIUNI ADMINISTRATIVE

- Pagina web: <http://moodle.cs.utcluj.ro>
 - Înregistrare
 - Sisteme de operare, Primăvara 2026, Seria 1
 - Cheia de înrolare - pt. grupa 3022**N**: **Grupa****N**.**SO.2026**
 - Atenție la modul în care apare numele: Firstname = Prenume
- Cerințe legate de cunoștințe preliminare
 - Cunoștințe de bază legate de sisteme de calcul
 - Programare C

APTITUDINI URMĂRITE

- Înțelegerea **rolului** jucat de SO în cadrul sistemelor de calcul
- Definirea și explicarea **conceptelor fundamentale** ale SO și a **funcționalităților** oferite de componentele acestora
- Înțelegerea și asimilarea principalelor **mecanisme** ale SO folosite în
 - Virtualizare
 - Concurență
 - Persistență
- Folosirea **apelurilor sistem Linux** în programe C
- Familiarizarea cu **comenzile de bază Linux**

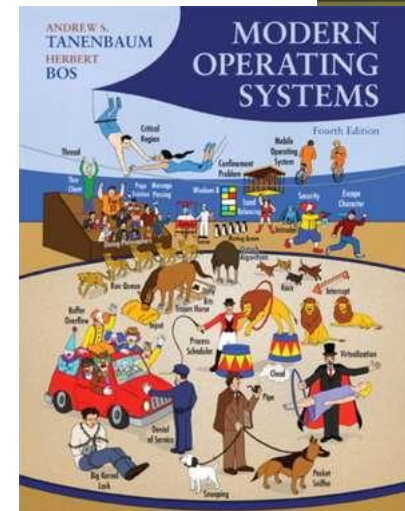
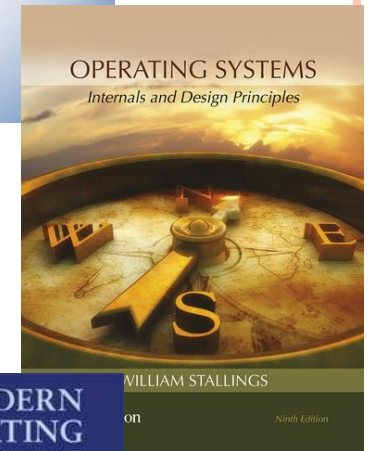
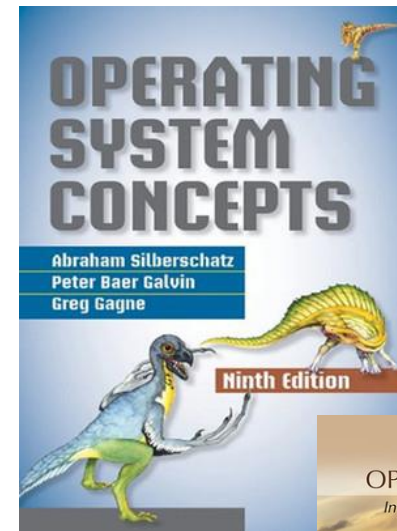
CUPRINS

- Introducere
- Structura sistemelor de operare
- Gestiuinea stocării
 - Sisteme de fișiere
 - Sisteme de I/E
- Gestiuinea proceselor
 - Procese
 - Threaduri
 - Planificare și sincronizare
 - Mecanisme de comunicare între procese (IPC)
- Gestiuinea memoriei
 - Memoria principală
 - Memoria virtuală
- Protecție și securitate

RESURSE BIBLIOGRAFICE

○ Bibliografie

- Silberschatz, Galvin, Gagne: Operating System Concepts, 9th Edition, 2012
 - Stallings, Operating Systems: Internals and Design Principles, 9th Edition, Pearson, 2021
 - Tanenbaum, Modern Operating Systems, 4th Edition, Pearson, 2015
 - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, online available at <http://pages.cs.wisc.edu/~remzi/OSTEP/>
- Alte resurse pe moodle

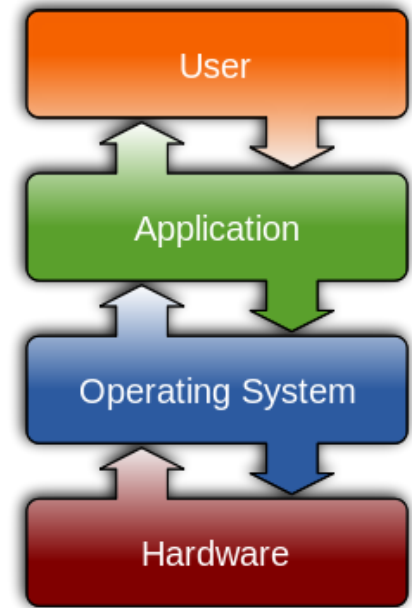


SISTEME DE OPERARE – DE CE?

- Pentru că sisteme de operare sunt **pretutindeni**
- Pentru că este **greu** să folosim calculatorul **fără** SO
- Pentru a înțelege **cum operează** (funcționează) sistemele de calcul
- Pentru a învăța cum se poate **gestiona complexitatea** prin abstracții potrivite
- Pentru a înțelege unele **decizii de proiectare**
 - Eficiență vs. simplitate, hw vs. sw, etc.
- Pentru a deveni un programator / inginer mai eficient

ROLUL SISTEMELOR DE CALCUL

- Calculatorul este un set de resurse pentru
 - **Rularea programelor**
 - stocarea
 - procesarea și
 - interschimbarea datelor
- **Componente**
 - **Hardware**
 - Procesorul, memoria, dispozitivele de I/E
 - **Sistemul de operare**
 - Controlează și coordonează utilizarea hardware-ului
 - **Programe aplicație**
 - Folosesc resursele sistemului pentru a soluționa problemele utilizatorului
 - Editoare de text, browsere, compilatoare, jocuri, etc.
 - **Utilizatori**
 - Umani, mașini, alte calculatoare



Figură preluată de pe wikipedia

ROLUL SO ÎN SISTEMELE DE CALCUL

- Arhitectura unui sistem de operare
 - Kernelul controlează hardware-ul
 - SO prezintă o interfață uniformă – independentă de detaliile hardwareului
 - Apeluri sistem
 - Programele aplicație și utilitare folosesc sistemul prin intermediul apelurilor sistem
 - Sunt lansate folosind interpretorul de comenzi sau prin interfata grafica

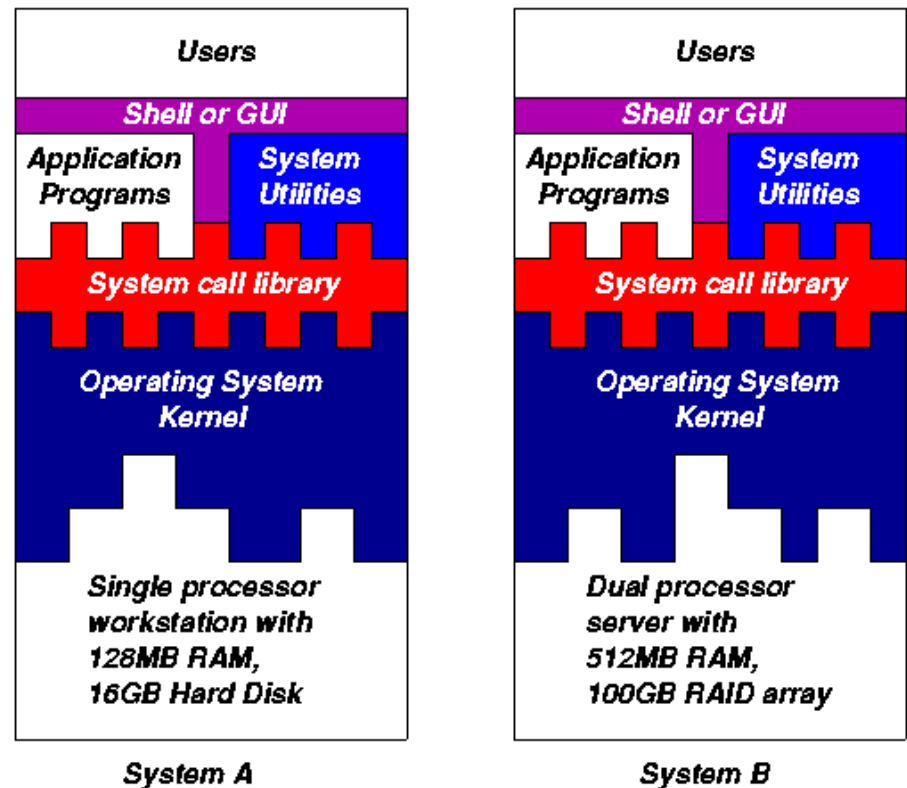


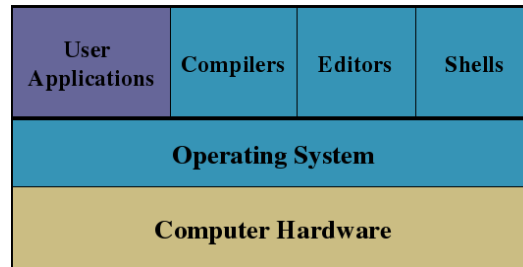
Fig. 1.1: General operating system architecture

CE ESTE SO?

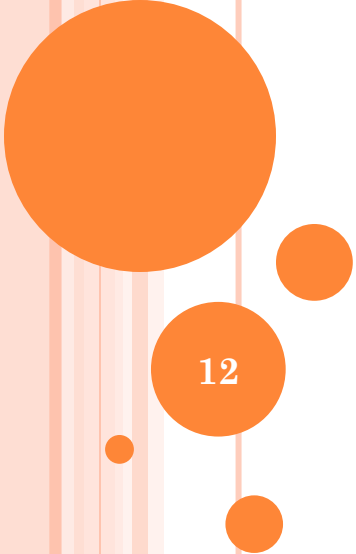
- Un software sistem
- Colectie de funcții și utilitare care
 - Gestionează resursele hardware
 - Permite rularea aplicațiilor utilizator prin care se folosesc resursele sistemului

SISTEME DE OPERARE – DEFINIȚIE

- Interfață între utilizator și hardware-ul sistemului
 - Responsabil pentru utilizarea corectă, eficientă și ușoară a sistemului
 - Furnizează o bibliotecă standard pentru aplicații
 - API – prin sute de apeluri sistem



- Dpdv logic se interpune între Hw și aplicații
- Dpdv fizic: este încărcat în memorie
 - similar cu aplicațiile utilizator
 - Rezidă în memorie și rulează tot timpul (kernel)
 - Pentru a permite rularea aplicațiilor

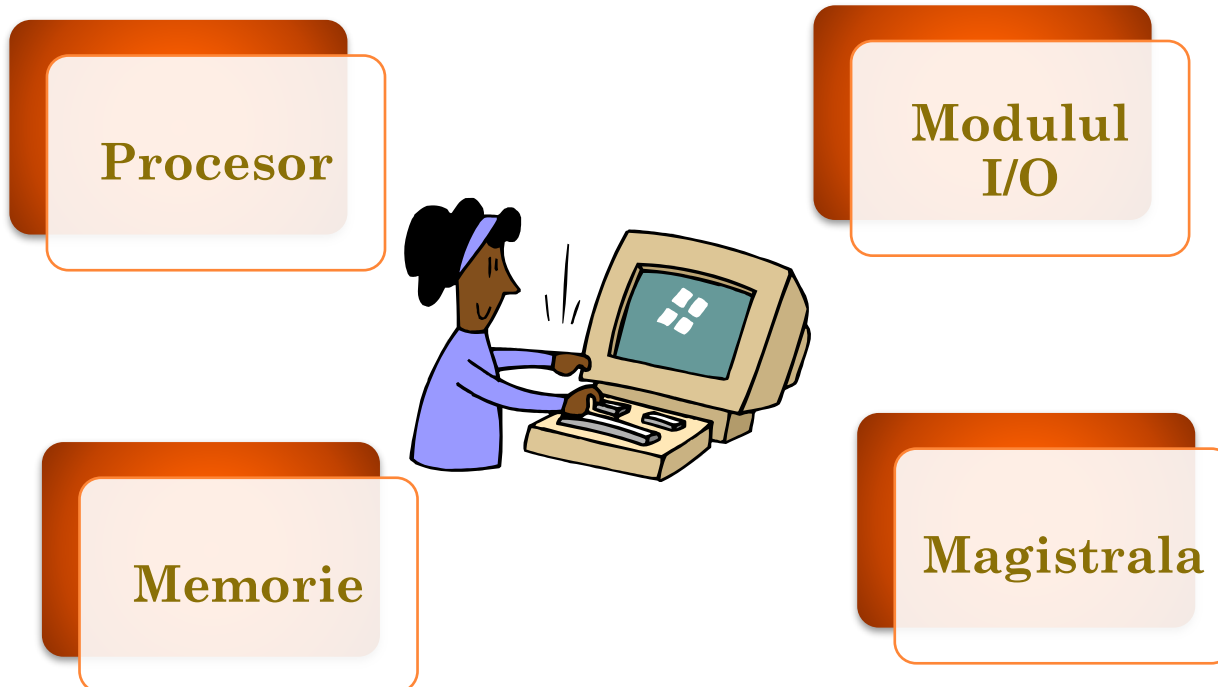


ORGANIZAREA SISTEMELOR DE CALCUL

12

ORGANIZAREA SISTEMELOR DE CALCUL

- Patru componente principale
 - Procesorul
 - Memoria internă
 - Dispozitivele de I/E
 - Magistralele



ORGANIZAREA SISTEMELOR DE CALCUL

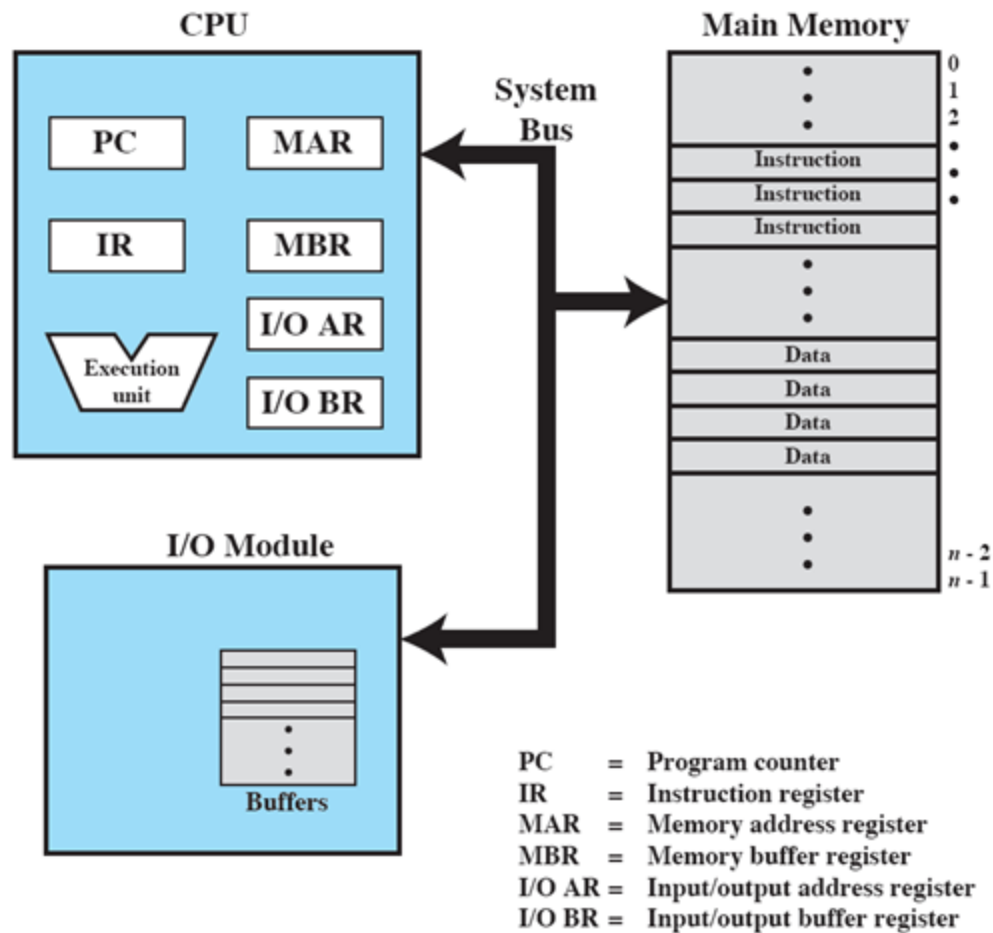
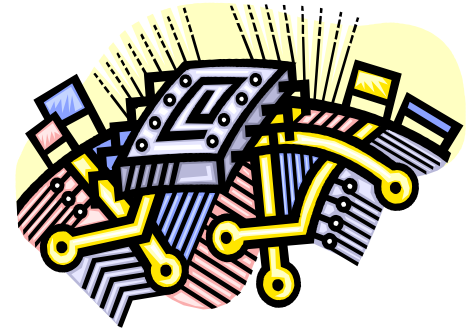


Figure 1.1 Computer Components: Top-Level View

PROCESORUL



- Rol: execuția de programe
 - Încarcă instrucțiuni din memorie, le decodifică, le execută
- Set de instrucțiuni (ISA – Instruction Set Architecture)
 - Set finit și bine definit de instrucțiuni – care este înțeles de procesor
- Regiștri
 - PC – Program Counter
 - IR – Instruction Register
 - SP - Stack Pointer, șamd.
 - Salvarea stării la schimbările de context
- Moduri (niveluri) de execuție
 - **Modul kernel** – privilegiat: acces deplin la toate resursele
 - Acces la tot setul de instrucțiuni
 - **Modul user** – neprivilegiat: fără acces direct la resursele hw
 - Acces doar la un set restrâns de instrucțiuni

PROCESORUL

- Programele constau dintr-un set de instrucțiuni stocate în memorie
- Execuția unei instrucțiuni simple

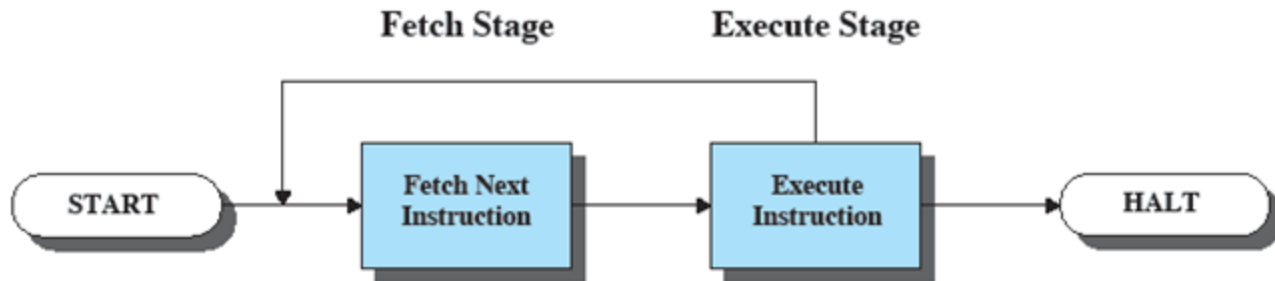
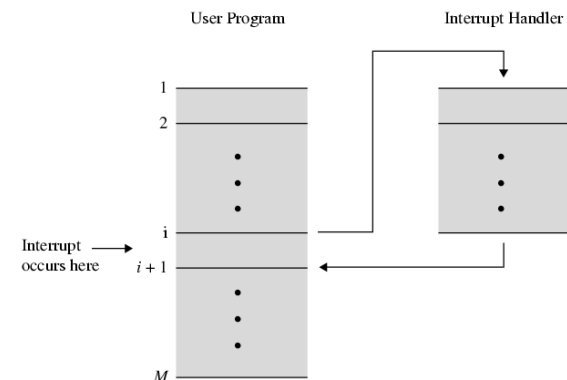
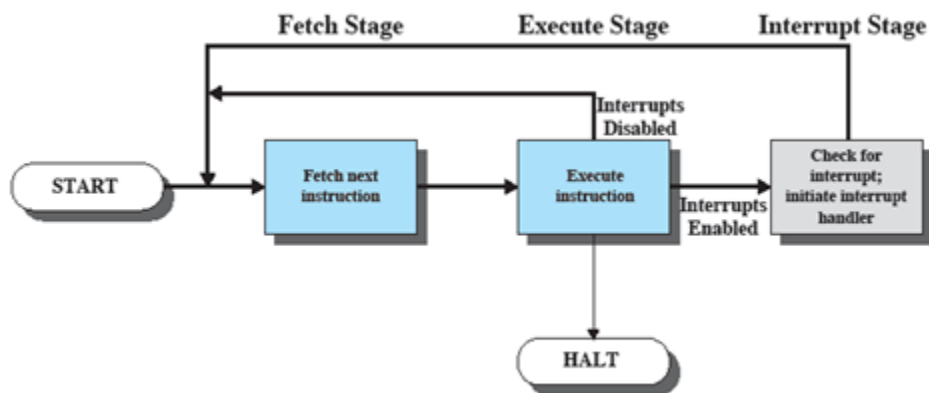


Figure 1.2 Basic Instruction Cycle

ÎNTRERUPERILE

- Esențiale pentru multiprogramare
- Întrerup execuția normală
 - Semnalează apariția unui eveniment
 - Procesorul deservește întreruperea executând un segment de cod
 - Rutina de tratarea întreruperii – în general parte din SO
 - Revenire la starea precedentă - se salvează valorile din regiștri și PC
- Scop
 - Creșterea eficienței
 - Ex. dispozitivele de I/E sunt mult mai lente decât procesorul
 - Procesorul este constrâns să aștepte dacă nu se folosesc întreruperi



Stallings, pg. 16

Figure 1.7 Instruction Cycle with Interrupts

FLUXUL CONTROLULUI CU ÎNTRERUPERII

Tip întrerupere

Cauză (ce o produce)

Rol / Scop

Excepții de program
(Program Exceptions)

Execuția unei
instrucțiuni invalide
sau problematice
(overflow, împărțire la
zero, instrucțiune
ilegală, adresă de
memorie invalidă)

Semnalează erori
apărute în timpul
execuției programului

**Întrerupere de
temporizator**
(Timer Interrupt)

Temporizator intern al
procesorului

Permite sistemului de
operare să execute
periodic sarcini
(planificare,
contabilizare timp etc.)

Întrerupere I/O
(I/O Interrupt)

Controler de
intrare/ieșire

Anunță finalizarea unei
operații I/O sau
apariția unei erori

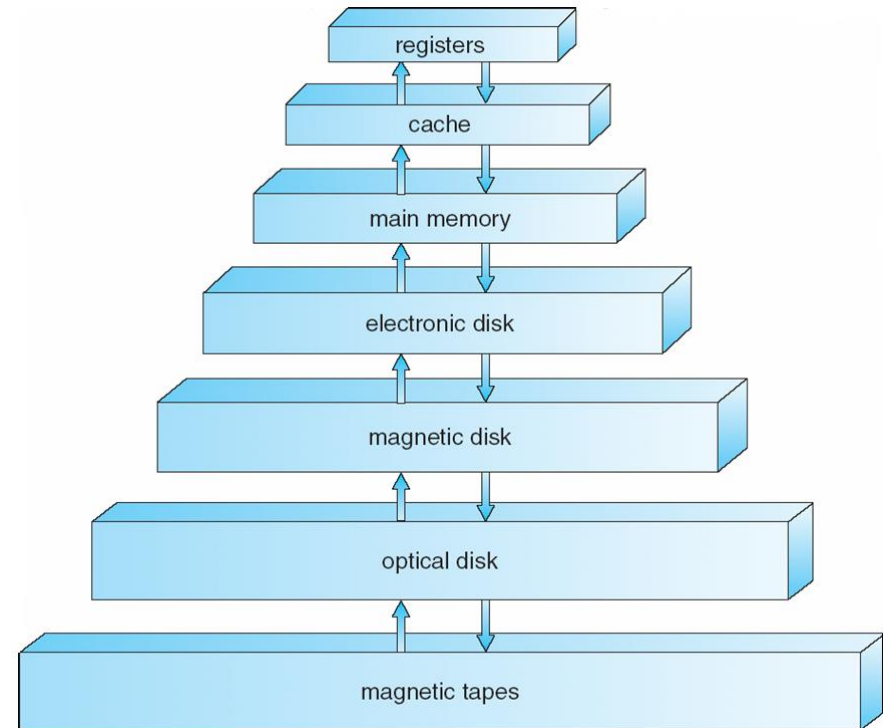
Defecțiuni hardware
(Hardware Failures)

Probleme fizice ale
echipamentului (cădere
de tensiune, eroare
memorie etc.)

Semnalează defecte
hardware ce necesită
tratare imediată

MEMORIA

- Rol: stocarea programelor care se execută pe procesor
- Constrângeri
 - Capacitate, viteză, cost
- Ierarhia de memorie
 - Regiștrii
 - Memoria cache
 - Memoria internă (RAM)
 - Volatilă
 - Memoria externă
 - Persistentă (HDD, SSD, DVD, etc.)
- De sus în jos
 - Descrește costul / bit
 - Crește capacitatea
 - Crește timpul de acces
 - Descrește frecvența de acces



Silberschatz, pg. 11

MEMORIA

○ Memoria cache

- Creșterea vitezei de acces la memorie
 - Informația utilizată este copiată - temporar - de pe mediul de stocare mai lent pe un mediu mai rapid
 - Motivație: viteza procesorului este mai mare decât cea a memoriei
- La accesul unor date - se verifică mai întâi cache-ul
 - HIT: informația este în cache – poate fi folosită direct
 - MISS: informația căutată nu este în cache – trebuie adusă

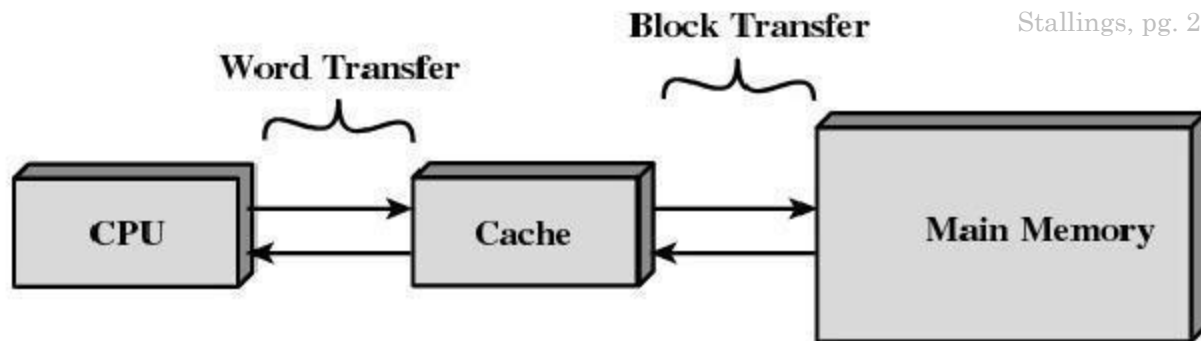


Figure 1.16 Cache and Main Memory
(p. 32)

DISPOZITIVELE DE I/E

- Componente

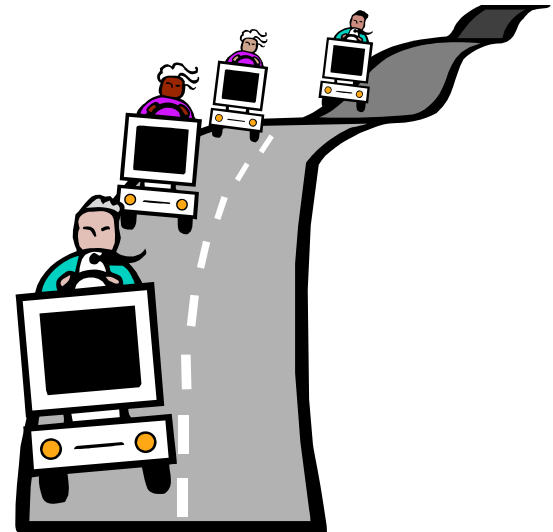
- Controlor
 - Controlează dispozitivul fizic
 - Primește comenzi de la SO (prin driver)
- Dispozitivul fizic

- Driverul de dispozitiv

- Furnizat de dezvoltatorul dispozitivului
- Inserat în SO – rulat în mod kernel

MAGISTRALE

- Furnizează modalitatea de comunicare între
 - procesor,
 - memorie și
 - dispozitivele de I/E



ROLUL SO

- Principiul de “infrumusețare” 😊 și control
 - Face ca hardware-ul să arate mai bine – mai prietenos
 - Abstracții potrivite, interfață simplă, ușor de folosit, independența față de detaliile hw
 - Controlează resursele și mediază utilizarea acestora
 - Partajare, multiplexare, monitorizare, protecție, raportare, etc.
- SO – similar cu un guvern
 - Face guvernul ceva util de unul singur?
 - Asigurarea bunei funcționări a resurselor și comunicarea/cooperarea între acestea
 - Prevenirea interferențelor dintre diferitele aplicații sau aplicație și SO
 - Crearea contextului în care aplicațiile pot fi rulate (corect, eficient, ușor)

ROLUL SO

○ Două viziuni

- De sus în jos: viziunea utilizatorului – **Mașină extinsă (sau mașină virtuală)**
 - Furnizor (abstrații utile)
 - De comoditate - prin servicii
 - De biblioteci standard – utilizarea resurselor în mod eficient
 - De mediu pentru dezvoltarea ușoară, rapidă a programelor
- De jos în sus: viziunea sistemului – **Manager de resurse**
 - Coordonator (similar cu un polițist rutier)
 - Gestiunea controlată a resurselor (alocare controlată)
 - Rezolvarea conflictelor pentru accesul la resurse (în timp și spațiu)
 - Prevenirea erorilor și utilizării incorecte a sistemului

SISTEME DE OPERARE – OBIECTIVE DE PROIECTARE

○ Abstractizarea

- Pentru ușurința în folosirea calculatorului
- Esențială în toate domeniile științei calculatoarelor
 - Programe mari – divizare în subprograme
 - limbaj de nivel înalt - limbaj de asamblare
 - limbaj de asamblare – porți logice
 - Proiectare procesor cu porți logice – nivel de tranzistor

○ Performanța

- Minimizarea încărcării introduse de SO și aplicații
 - Costuri în timp și spațiu

○ Protecție

- Între aplicații și între SO și aplicații
- Soluție: izolarea

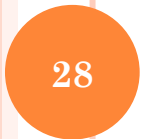
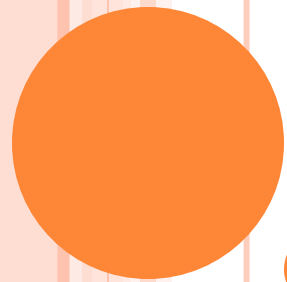
○ Fiabilitate

- SO rulează tot timpul
 - Dacă eșuează -> tot sistemul eșuează

○ Eficiență energetică

○ Mobilitate

○ și multe altele



28



CONCEPTE SO FUNDAMENTALE

CONCEPTE SO - PROCESUL

- Rolul principal al SO este rularea proceselor
- Un proces este un **program în execuție**
 - Unitatea de lucru în cadrul sistemului
- Program vs. Proces
- Fiecare proces are nevoie de resurse pt rulare
 - Procesor, memorie, i/e, fișiere
 - Date de inițializare
- La terminare resursele reutilizabile trebuie eliberate
- Un proces (cu un singur thread – fir de execuție) are un singur PC (Program counter)
 - Indică locația următoarei instrucțiuni
 - Executare secvențială a instrucțiunilor
- Sistemul are mai multe procese care rulează concurent pe unul sau mai multe procesoare
 - Procese utilizator
 - Procese sistem
 - Concurența – prin multiplexarea procesorului între procese / threaduri

CONCEPTE SO - PROCESUL

○ Fiecare proces are

○ Spațiu de adrese

- Spațiu de memorie din/în care procesul poate citi/scrie
- Include
 - Segmentul de cod – instrucțiunile din executabilul programului
 - Segmentul de date – datele cu care lucrează programul
 - Stiva

○ Mediul necesar rulării programului

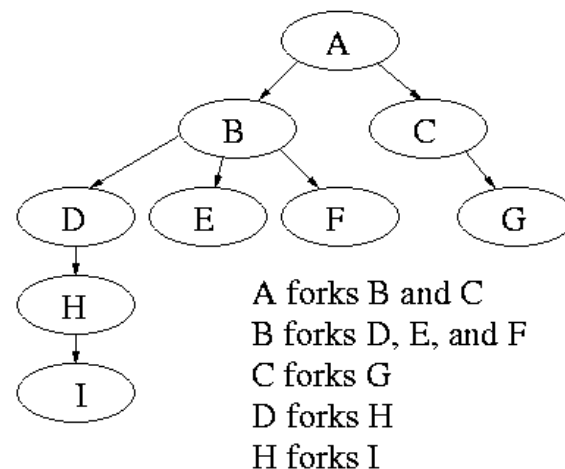
- Pentru înțelegere mai ușoară – ne raportăm la multiprogramare
 - Periodic SO decide să oprească un proces și să-i aloce procesorul altui proces
 - La suspendare trebuie salvate toate informațiile necesare restartării procesului (ex. fișiere deschise, pointer la pozițiile curente, etc.)
 - Tabela de procese – tablou de structuri, câte una pentru fiecare proces
 - SET DE REGISTRI
 - PC (PROGRAM COUNTER), SP (STACK POINTER), ETC.

CONCEPTE SO - PROCESUL

○ Gestiunea proceselor

- SO oferă mecanisme pentru
 - **Crearea și terminarea** unui proces

- Un proces poate crea unul sau mai multe procese
- Relație părinte - fiu
- Ierarhie de procese



- **Suspendarea și reluarea** unui proces
- **Sinconizarea** proceselor
- **Comunicare** între procese
- **Detectarea și tratarea interblocărilor** (deadlock)

CONCEPTE SO - MEMORIA

- Toate datele sunt în memorie
 - înainte de procesare
 - după procesare
- Instrucțiunile trebuie să fie încărcate în memorie pentru a fi executate
- Gestiunea memoriei
 - Ce este în memorie și când
 - Scop: **optimizarea utilizării procesorului și reducerea timpului de răspuns**
- SO oferă mecanisme pentru
 - **Alocarea** memoriei
 - Alocarea sau eliminarea unui proces din mem
 - Alocarea datelor pentru procesare
 - Determinare - **ce parte** a memoriei este **folosită** și de **cine**
 - **Memorie virtuală**
- **Izolarea proceselor**
 - Prevenirea interferenței dintre procese
 - Protecție și controlul accesului la partajarea memoriei

CONCEPTE SO – FIȘIERUL

- Utilitatea dispozitivelor de stocare
 - date care nu încap în memorie sau
 - Date care trebuie păstrate perioadă mai lungă de timp (persistent)
- Gestiunea stocării - viteza întregului sistem depinde de sistemul de stocare și de algoritmi folosiți
- SO oferă mecanisme pentru
 - Gestiunea spațiului liber
 - Alocarea spațiului de stocare
 - Planificarea operațiilor pe disc
- Fișierul
 - **unitatea fundamentală în stocarea persistentă a datelor**
- Sistemul de fișiere
 - fișiere și directoare
 - structura ierarhică

SUBSISTEME DE I/E

- Ascunderea complexității și specificității dispozitivelor hardware
 - Una din rolurile importante ale SO
- Subsistemul de I/E este responsabil pentru
 - Gestiunea memoriei pentru i/e și include
 - Gestiunea zonelor tampon (buffering) – stocarea temporară a datelor pe timpul transferului
 - Gestiunea cache-ului – stocarea datelor în memorie mai rapidă pentru îmbunătățirea performanței
 - Spooling – suprapunerea ieșirii unui job cu intrarea altuia
 - Interfața generală cu dispozitivul (driver)
 - Gestiunea driverului pentru fiecare dispozitiv

PROTECȚIE ȘI SECURITATE

○ Protecție

- Mecanism pentru **controlul accesului** la resurse

○ Securitate

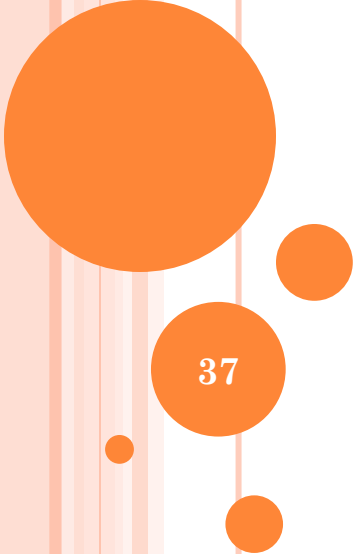
- Mecanisme pentru **apărarea sistemului** împotriva atacurilor externe sau interne

○ Metode

- Diferențiere între utilizatori, grupuri de utilizatori
 - id, gid
- Asociere id cu fișiere
- Drepturi de acces
- Evitarea atacurilor de escaladare privilegiată

Operațiile SO

- SO este **controlat prin întreruperi**
 - Hardware – semnal direct la procesor prin magistrală
 - Software – se mai numește excepție (sau trap)
 - Ex. cerere de serviciu de la SO, împărțirea la zero, etc.
- Probleme care apar cu multiprogramarea
 - Un job **monopolizează procesorul** prin buclă infinită
 - Un job **modifică zona de memorie** a altui job sau chiar a SO
- Soluție: **operare duală**
 - **Mod utilizator**
 - **Mod kernel**
 - Bit de mod – indică în ce mod rulează sistemul
 - Unele operații se pot efectua doar în mod kernel
 - Operații privilegiate (acces la resursele sistemului)



SERVICII OFERITE DE SO

37

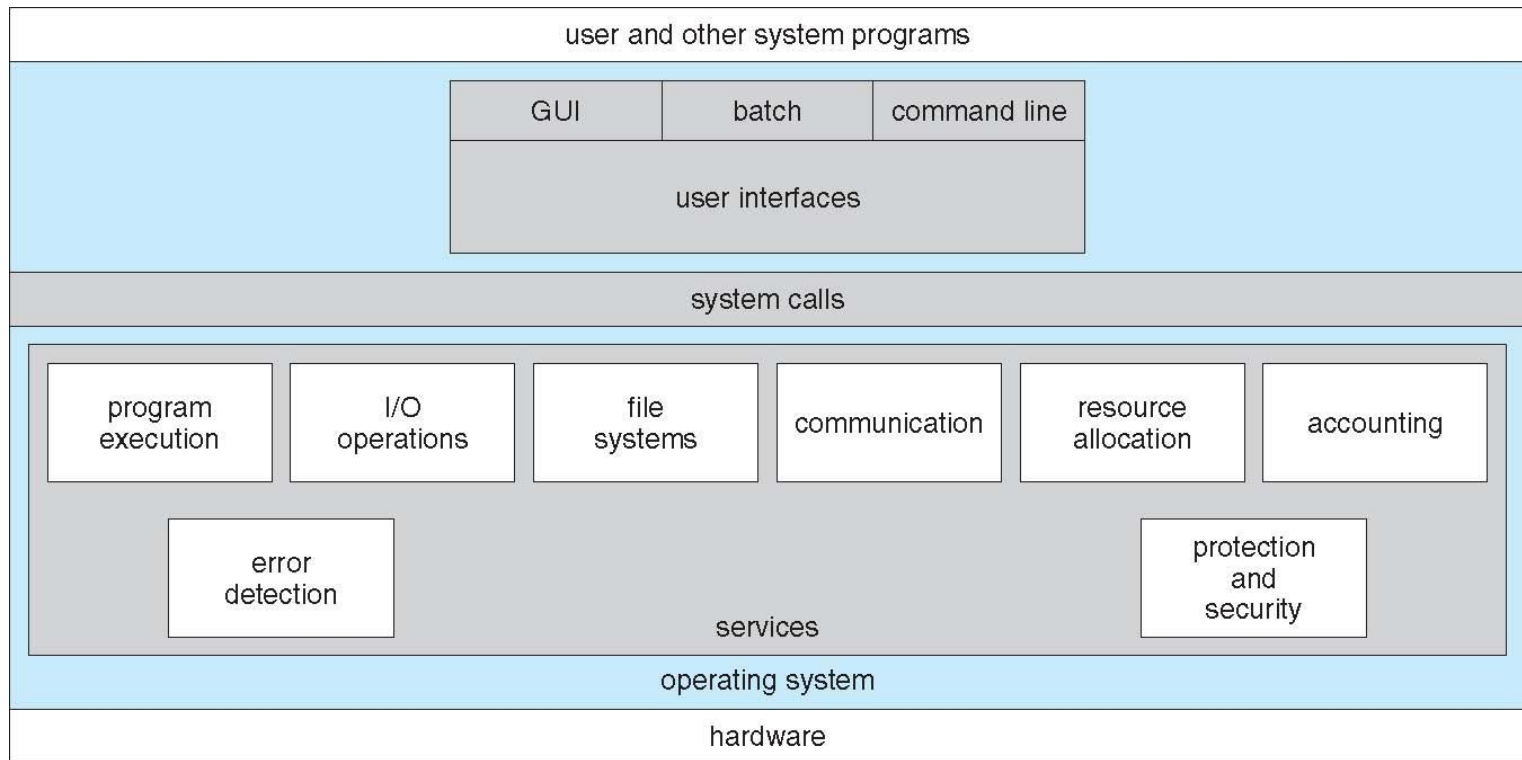
SERVICII OFERITE DE SO

- SO oferă un **mediu pentru dezvoltarea și rularea programelor**
- Unele **servicii** oferă funcții utile utilizatorului pentru **interacțiunea cu sistemul**
 - **Interfață utilizator** - CLI (command line interface), GUI (graphics user interface), Batch
 - Pentru dezvoltarea programelor proprii / acces la utilitarele/serviciile sistem
 - **Executarea de programe**
 - Încărcarea în memorie și rularea programelor
 - **Accesul la dispozitivele de I/E**
 - Deservirea cererilor de acces la fișiere sau dispozitive – interfață uniformă
 - Gestiunea sistemului de fișiere
 - **Detectarea erorilor**
 - Erori pot apare de oriunde (hw și sw)
 - Tratarea erorilor prin acțiuni potrivite pentru a asigura funcționarea corectă și consistentă
 - **Comunicare**
 - Între procesele aceluiași sistem sau prin rețea
 - Prin memorie partajată, transmitere de mesaje, etc.

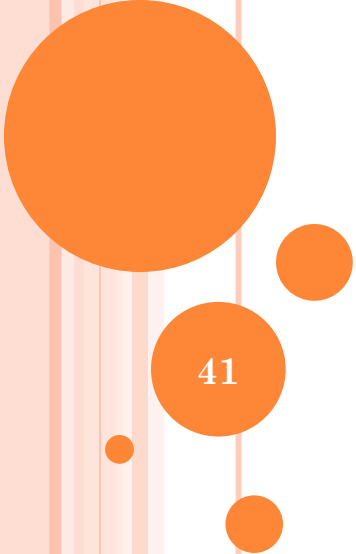
SERVICII OFERITE DE SO

- Alte servicii care asigură **operarea eficientă** a sistemului prin **partajarea resurselor**
 - **Alocarea resurselor**
 - Mai multe joburi concurente-> resurse alocate corect
 - **Evidența utilizării resurselor**
 - Ce resurse și cât din resurse folsește fiecare utilizator
 - **Protecție și securitate**
 - Protejarea accesului la informațiile utilizatorilor
 - Controlul accesului la resurse
 - Procesele concurente să nu interfereze în mod malițios
 - Autentificare
 - Multe alte servicii și mecanisme pentru securizare de mediul exterior și interior

SERVICII OFERITE DE SO



Silberschatz, pg. 50

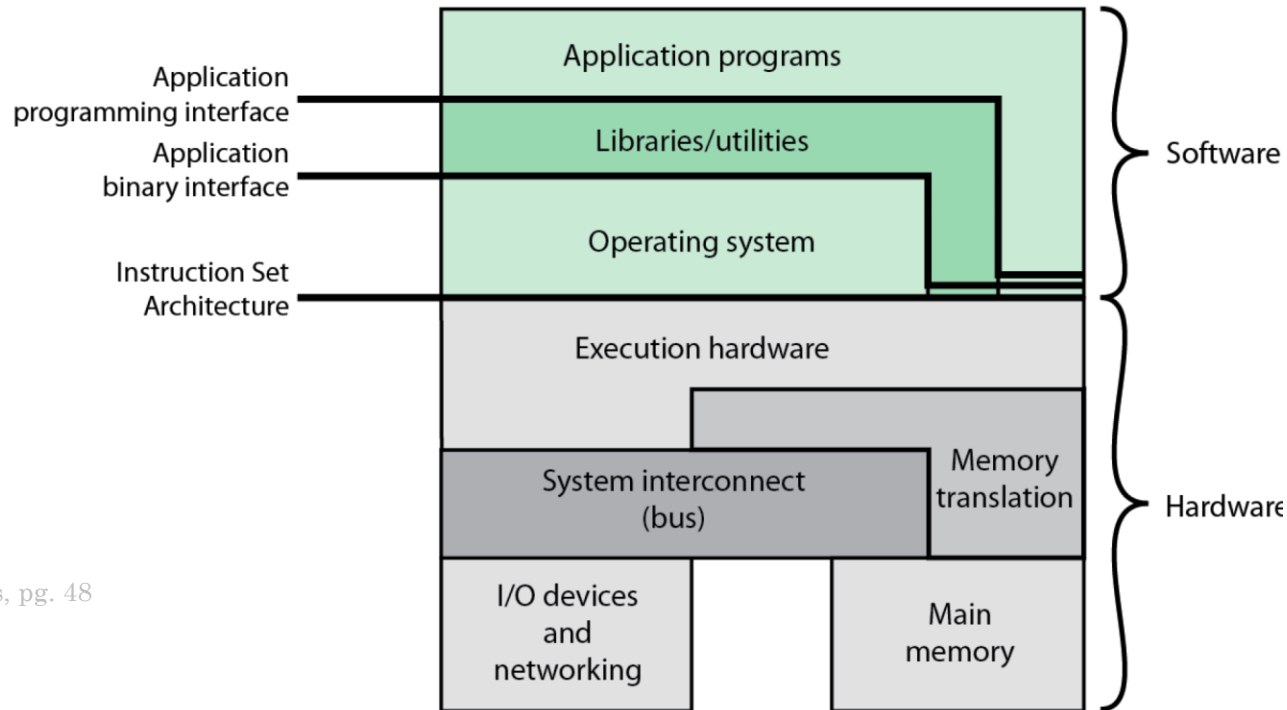


APELURI SISTEM

41

SISTEME DE OPERARE – INTERFEȚE

- Interfața cu hardware-ul
- Interfața cu utilizatorul



Stallings, pg. 48

Figure 2.1 Computer Hardware and Software Infrastructure

APELURI SISTEM

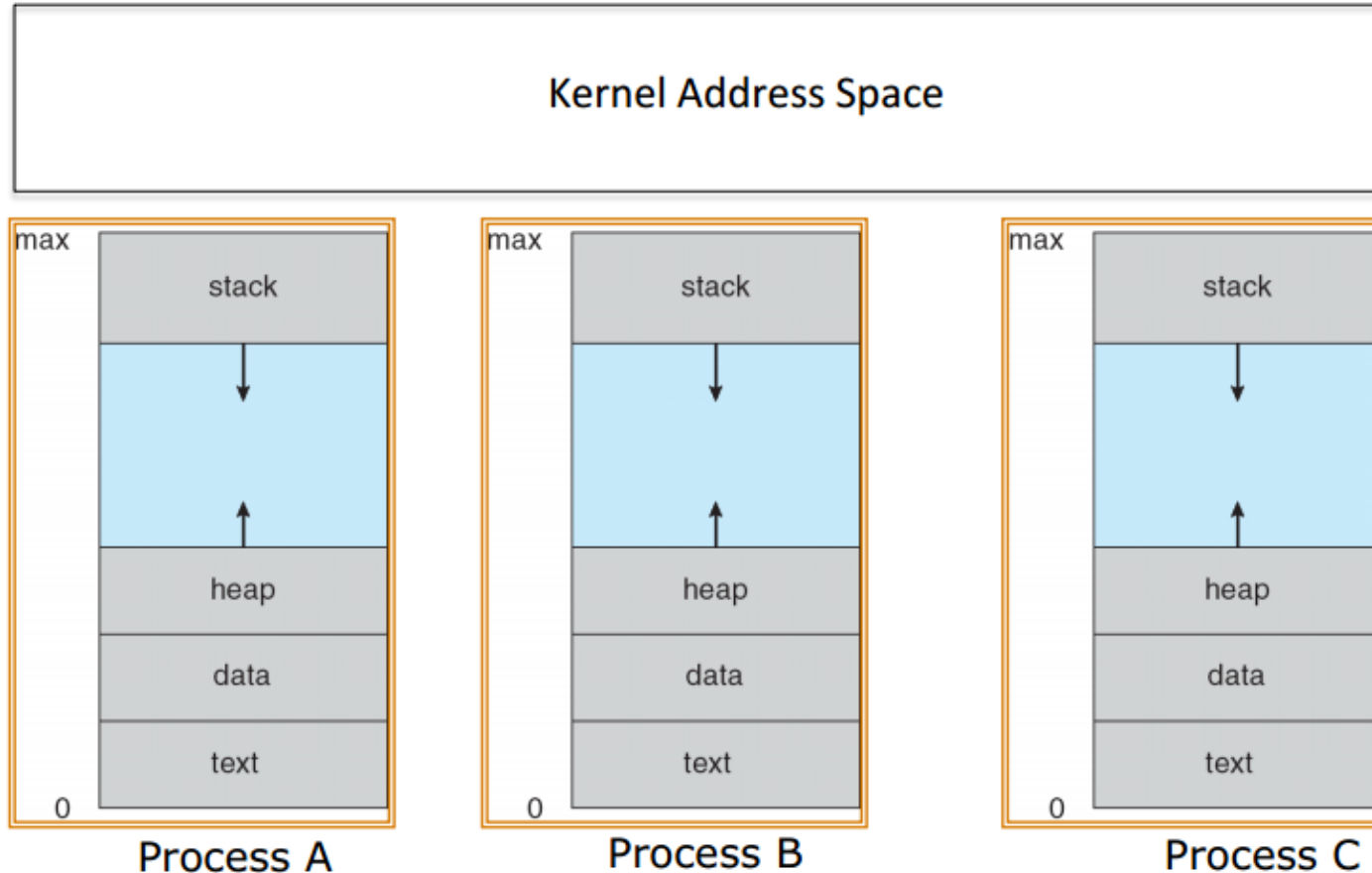
○ Interfața dintre SO și aplicațiile utilizator

- Definită de un set de instrucțiuni extinse – **apeluri sistem**
 - Diferă de la un SO la altul
 - Conceptele care stau la bază sunt similare
- Pentru a înțelege ce face SO trebuie să studiem apelurile sistem
 - Două modalități
 - Prezentarea unor generalități vagi
 - Prezentarea în cadrul unui sistem specific – varianta aleasă – Linux

○ Apeluri sistem

- Mecanism prin care aplicațiile utilizator pot solicita servicii de la SO
- Sunt **executate în mod kernel** (privilegiat) – au acces la resurse
 - Argumentele trebuie atent verificate (să nu “convingă” kernelul să facă ceva ce un proces utilizator nu ar trebui să facă)

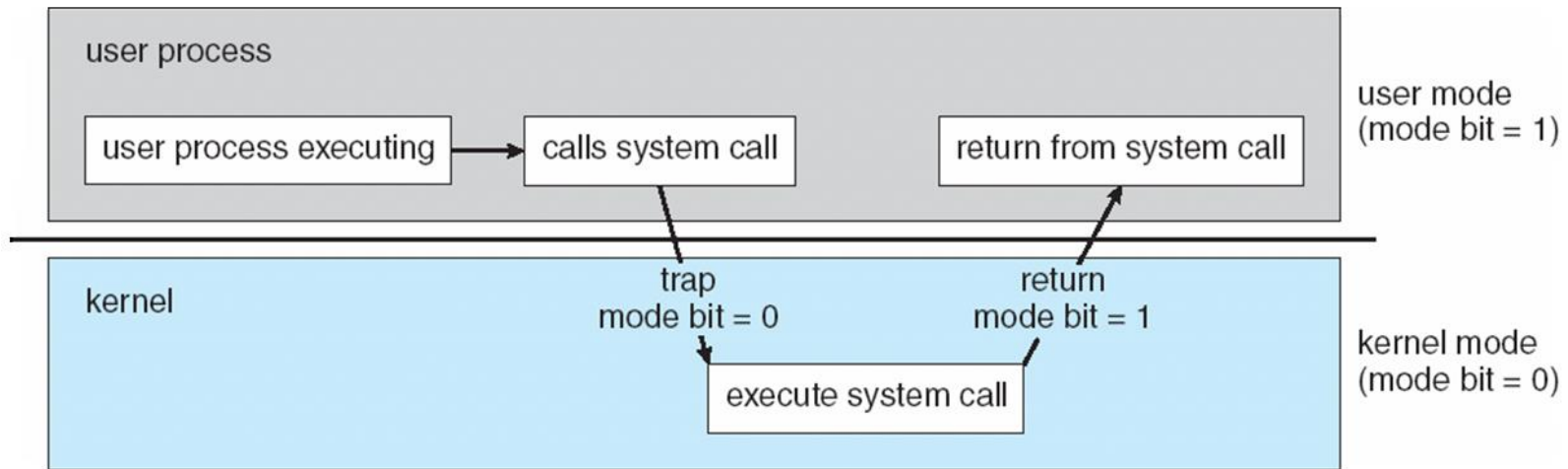
APELURI SISTEM



- Procesele “trăiesc” în spațiul de adrese propriu
 - Nu pot accesa direct lumea exterioară
- Comunică prin **apeluri sistem**

APELURI SISTEM

- Tranziția din modul utilizator în modul kernel

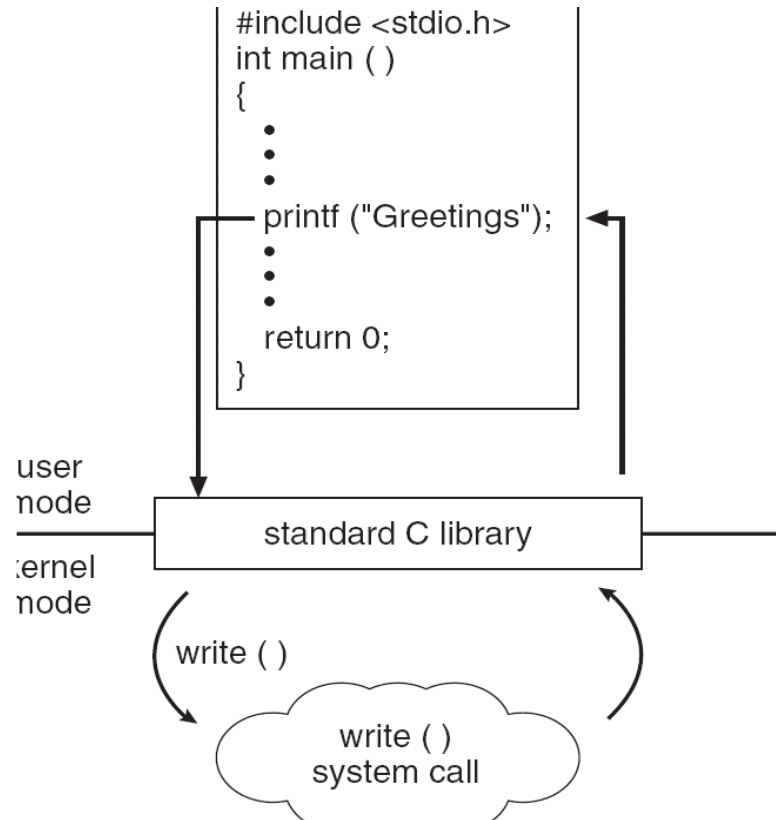


Silberschatz, pg. 21

- Necesită mecanism special de tip *trap* pentru a trece la nivel privilegiat de acces la serviciile sistemului de operare
 - pentru a „sări” din spațiul utilizatorului (codul aplicației) în spațiul kernel (codul sistemului de operare)
 - Apeluri de sistem:** mod voluntar, sincron și controlat
 - Excepții** (și tratare): mod involuntar, asincron și necontrolat

APELURI SISTEM

- Program C care apelează funcția printf()
 - Care invocă apelul sistem write()

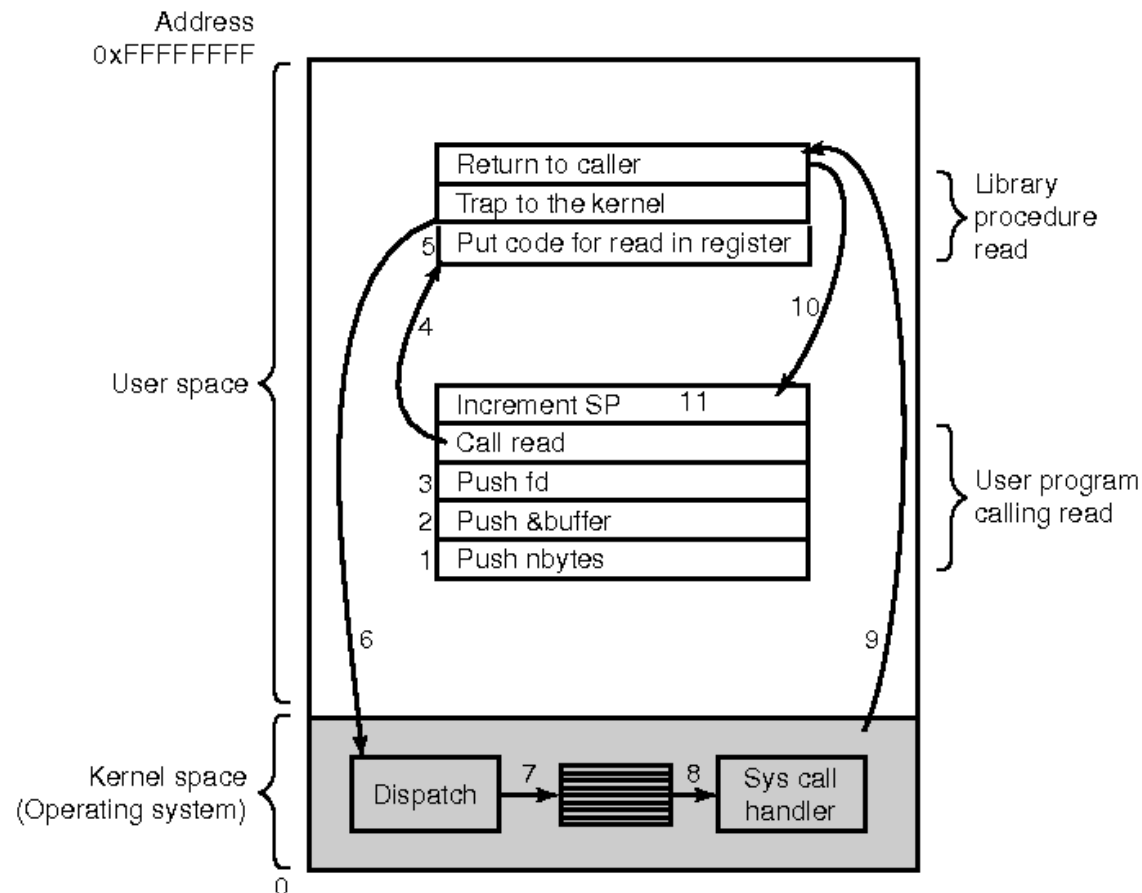


Silberschatz, pg. 62

APELURI SISTEM

Program C care apelează funcția read()

```
count= read(fd, buffer, nbytes);
```



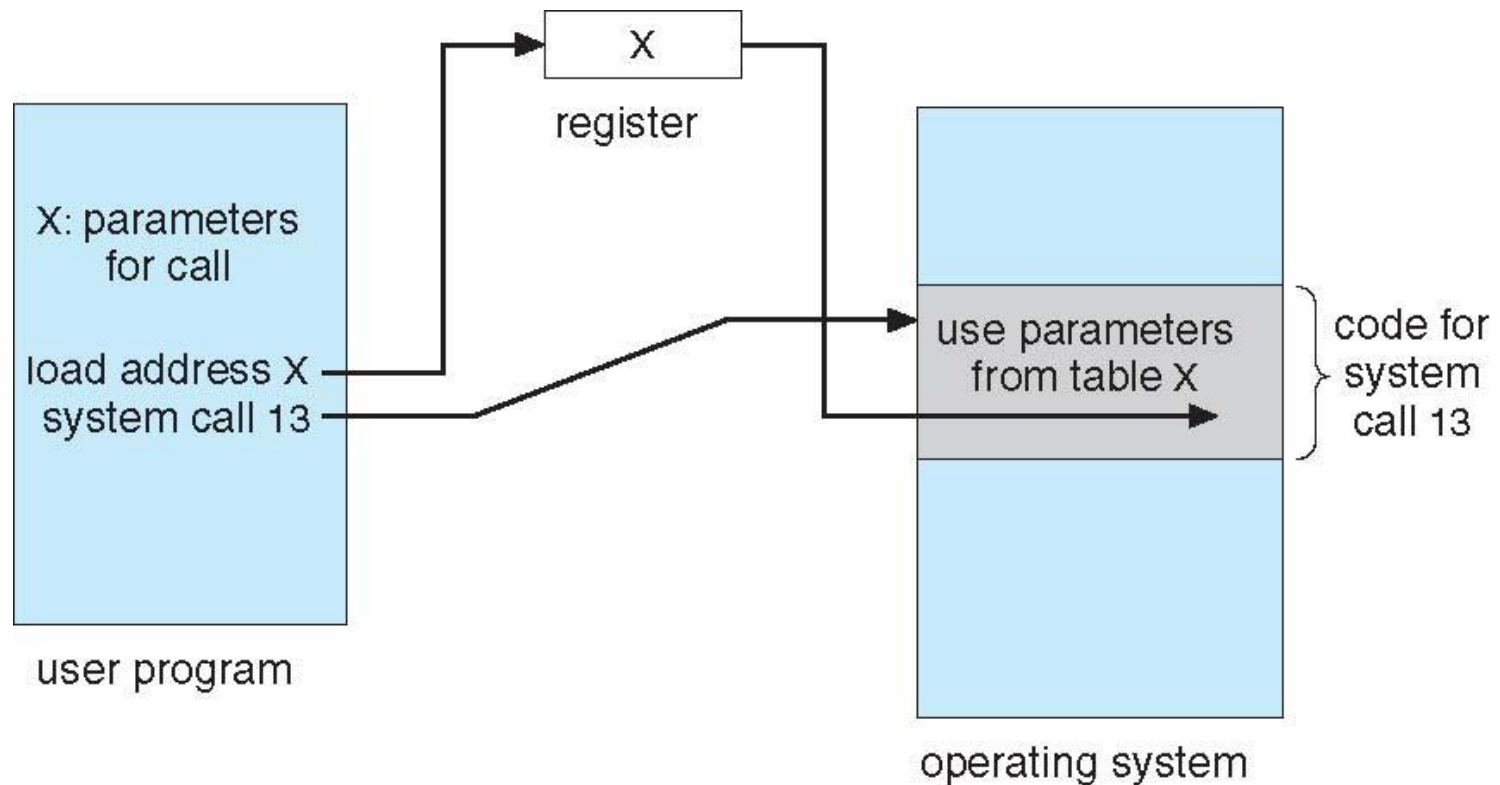
Tanenbaum, pg. 46

APELURI SISTEM

- Scrise în general în C (C++)
- Accesate în general prin API (Application Program Interface) de nivel înalt
 - Ex. Win32 API pentru Windows
POSIX API (UNIX, Linux, Mac OS X, etc.)
Java API pentru JVM (Java Virtual Machine)
- De multe ori sunt necesare mai multe informații decât numele apelului sistem
 - Tipul și numărul de informații diferă de la un SO la altul
 - Transmiterea parametrilor către SO
 - Transmitere prin regiștri
 - Problema: numărul argumentelor poate fi mai mare decât numărul de regiștri disponibili
 - Transmitere prin tabel
 - Parametrii sunt puși într-un tabel și se transmite adresa de memorie printr-un registru (Linux)
 - Prin stivă (push, pop)

APELURI SISTEM

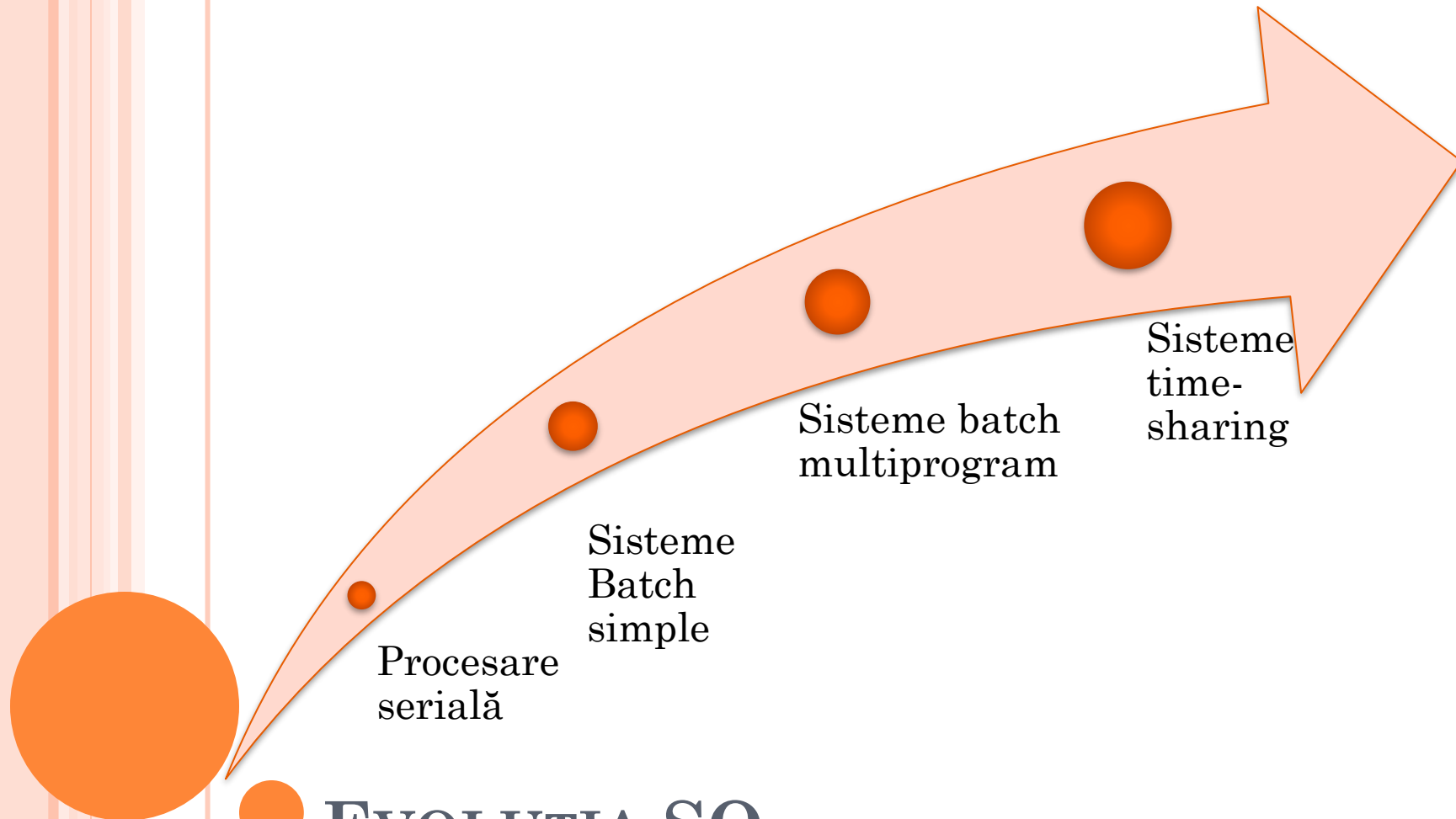
- Transmiterea parametrilor prin tabel



APELURI SISTEM

○ Exemple de apeluri sistem pe Windows resp. Linux

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



EVOLUȚIA SO

Stadii în evoluție

Ca orice sistem construit de oameni, ideile bune se acumulează în timp

EVOLUȚIA SISTEMELOR DE OPERARE

- Evoluția în timp se datorează
 - Evoluției hardware-ului
 - Un hw complex necesită un sistem de operare pe măsură
 - Utilizare eficientă
 - Ascunderea complexității
 - Integrarea de noi servicii

Evoluția hardware-ului

Noi tipuri de hardware

Noi servicii

Corecții aduse SO

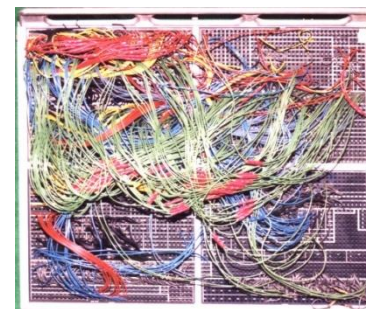
EVOLUȚIA SO

- Prima generație (1945 - 1955)

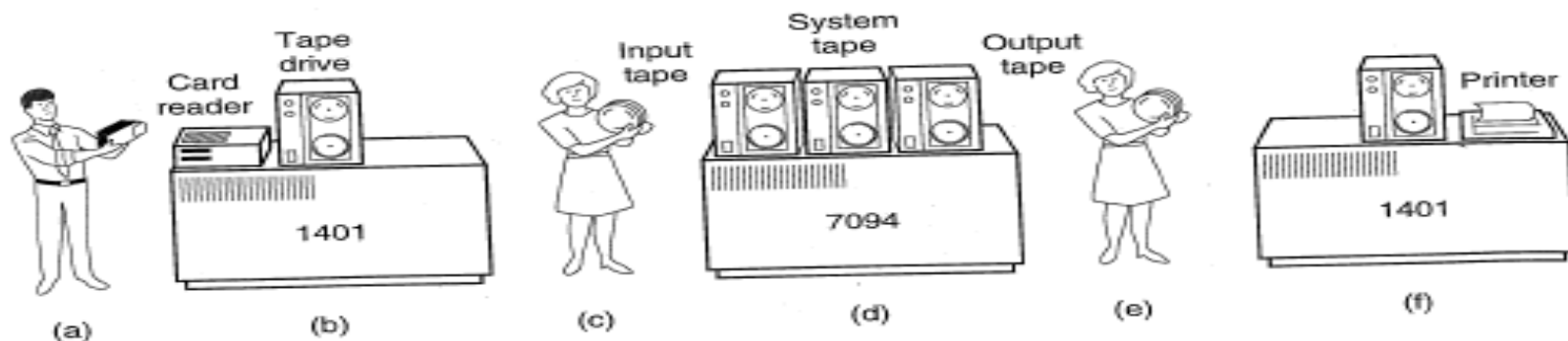
- Tuburi cu vid și plăci de conexiuni
- Procesare serială

- A doua generație (1955 - 1965)

- Tranzistoare, sisteme batch
- Mai multe programe pe disc



vacuum tube from the early 1900's



- A treia generație (1965 - 1980)

- Circuite integrate și multiprogramare
- Interacțiune cu utilizatorul – time sharing

Tanenbaum, pg. 8

- A patra generație (1980 - prezent)

- Calculatorul personal – orientat pe interfața utilizator și eficiență
- Rețele și Internetul
- Sisteme paralele, portabile etc.

EVOLUȚIA SISTEMELOR DE OPERARE

○ Etapa 1

- Hardware-ul este scump, oamenii sunt ieftini

- Faze

- Procesare serială
- Procesare batch
- Sisteme cu multiprogramare

PROCESARE SERIALĂ – 1945-1955

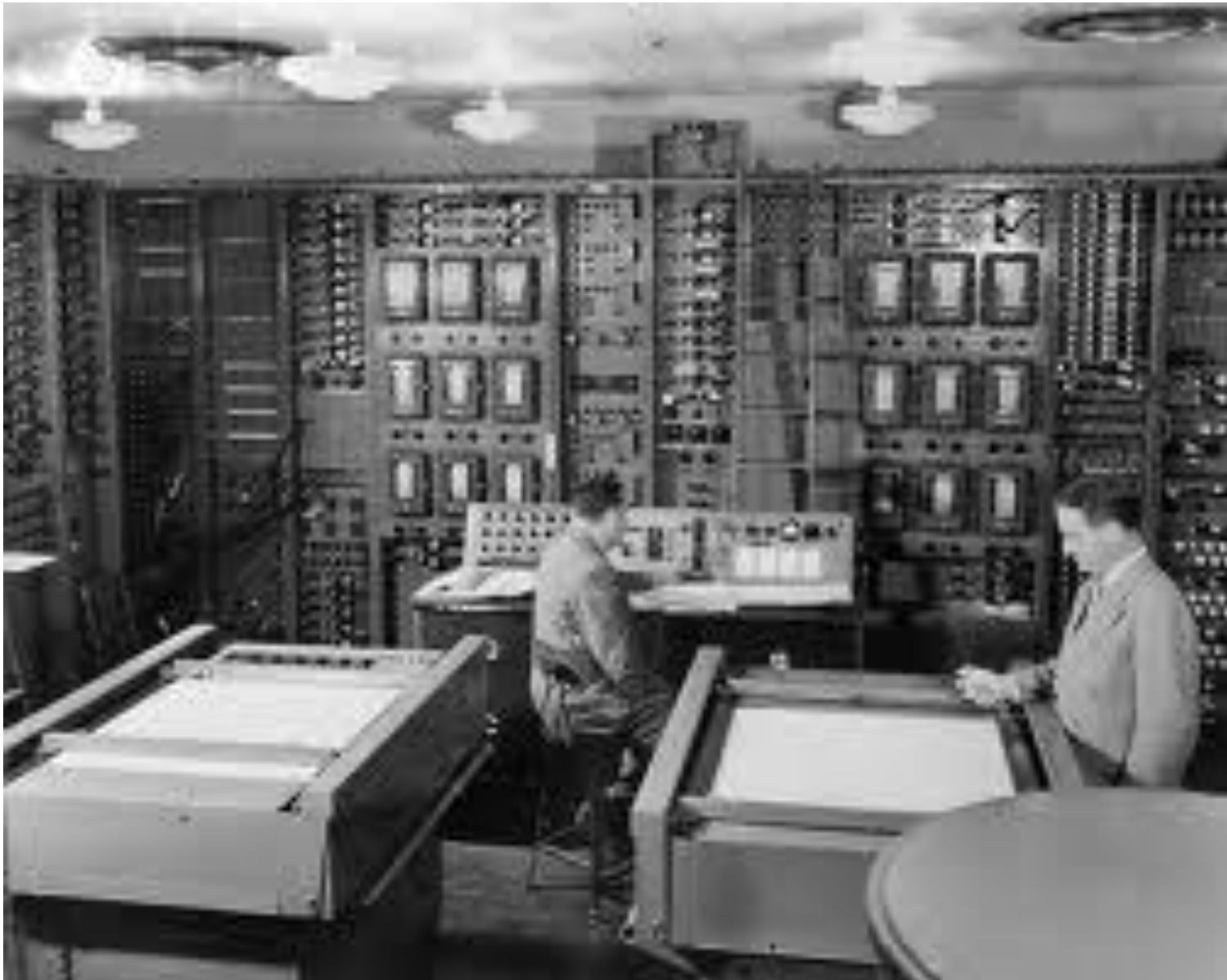
○ Primele calculatoare

- Fără SO
- Programatorul interacționează direct cu hw
- Sistemul rulează un singur program în orice moment
- Accesul la calculator se face serial
 - Un programator după celălalt

○ Probleme

- Utilizare inefficientă a resurselor
 - HW – ex. procesorul așteaptă terminarea operațiilor pe disc
 - Utilizatorii umani – trebuie să-și aștepte rândul
- Planificare
 - Înscriere și rezervarea unui interval fix (dacă e prea mult timp, sau prea puțin?)
- Setup
 - Pregătirea necesită timp îndelungat (încărcarea compilatorului, a codului sursă, salvarea programului compilat, încărcarea executabilului, editarea legăturilor)

PROCESARE SERIALĂ – 1945-1955



SISTEME BATCH SIMPLE – 1955- 1965

- Primele calculatoare erau foarte scumpe și foarte puține (mainframe-uri)
 - Maximizarea utilizării procesorului
- Introducerea **bibliotecilor standard**
 - funcții utilizate frecvent
 - Ex. API pentru accesul la dispozitive (scriere, citire, etc.)
- **Monitor**
 - Monitorul controlează execuția programelor
 - Rezident în memorie
 - Citește jobul următor
 - Predă controlul programului
 - Programul redă controlul monitorului
 - Îndrumarea monitorului prin Job Control Language instrucțiuni care indică
 - Compilatorul care trebuie folosit
 - Datele care trebuie procesate, etc.
 - Programatorii nu mai aveau acces direct la procesor
 - Joburile sunt predate operatorului uman care le stivuiește și la plasează în dispozitivul de intrare

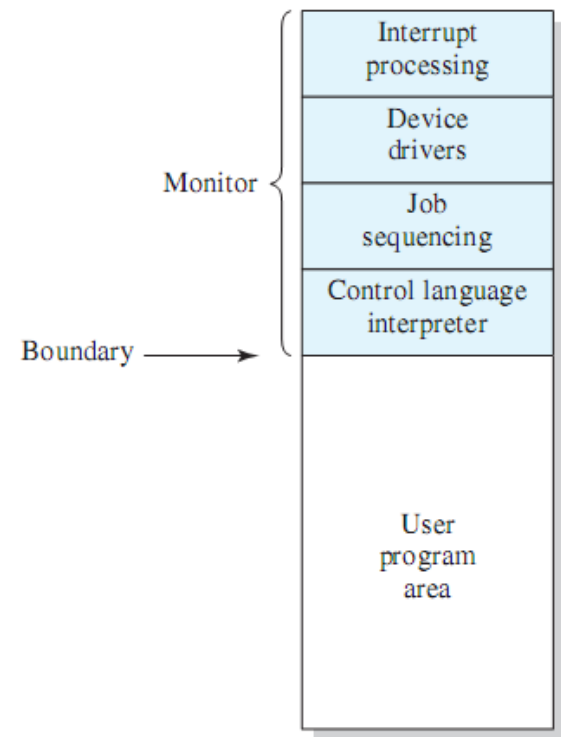
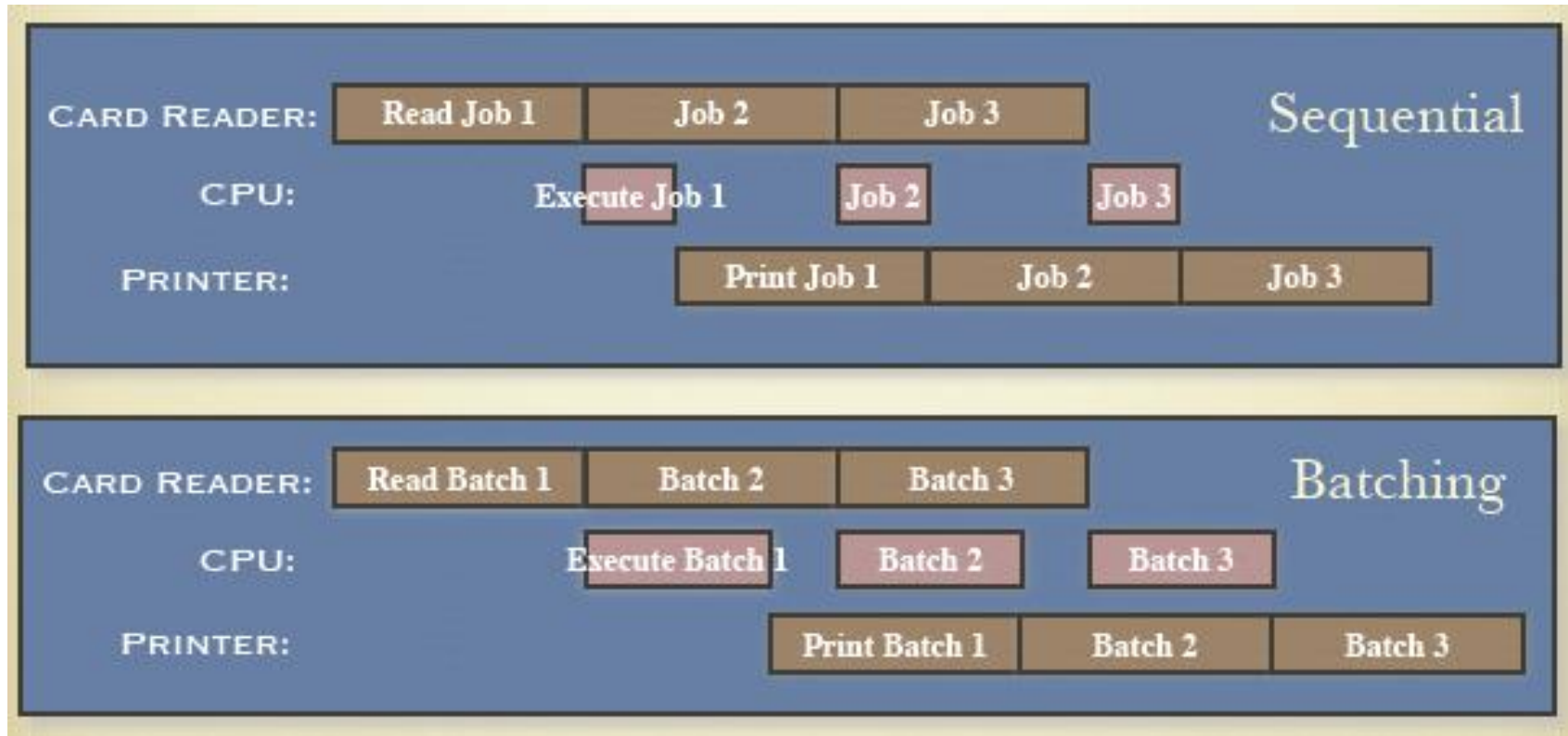


Figure 2.3 Memory Layout for a Resident Monitor

SISTEME BATCH SIMPLE – 1955- 1965



SISTEME BATCH SIMPLE – 1955- 1965

○ Concepte introduse

○ Protecția memoriei pentru Monitor

- Aplicația utilizator să nu poată modifica zona de memorie unde se află monitorul

○ Temporizare

- Previne acapararea procesorului de către un singur job

○ Instrucțiuni privilegiate

- Instrucțiuni care pot fi efectuate doar de monitor
- Apeluri sistem (vs. apeluri de procedură)
- Mod kernel – mod utilizator
 - Transfer protejat al controlului

○ Întreruperi (trap)

SISTEME BATCH MULTIPROGRAM – 1965 - 1980

○ Motivație

- Procesorul este frecvent în așteptare
 - Chiar dacă execuția joburilor este automatizată
- Dispozitivele de I/E sunt mult mai lente decât procesorul

Read one record from file	15 μ s
Execute 100 instructions	1 μ s
Write one record to file	<u>15 μs</u>
TOTAL	31 μ s

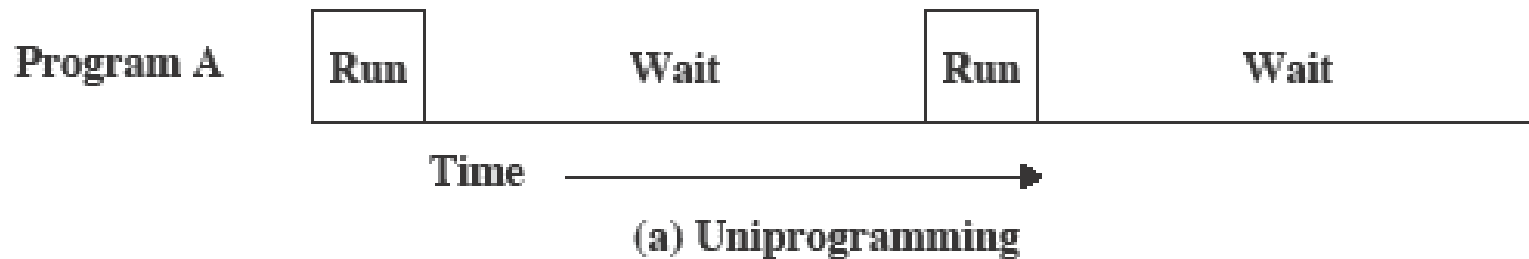
$$\text{Percent CPU Utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

Figure 2.4 System Utilization Example

Stallings, pg. 56

DE LA UNIPROGRAMARE

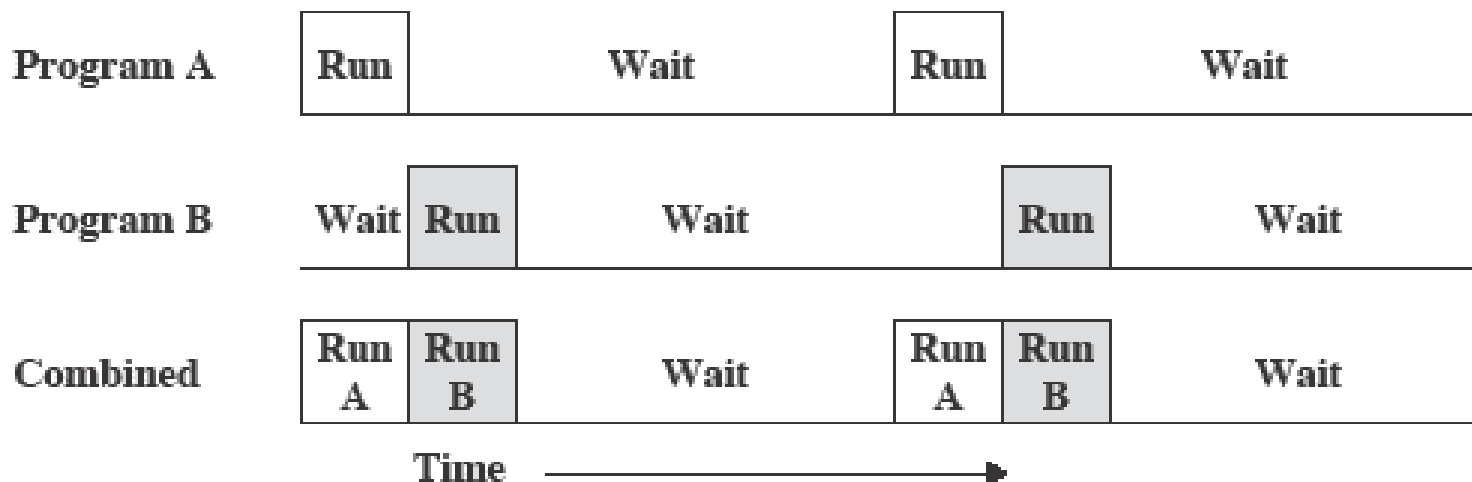
Stallings, pg. 57



- Procesorul execută instrucțiuni pentru un interval de timp până ajunge la o instrucțiune I/Ė
- Așteaptă terminarea instrucțiunii i/e
- Trece mai departe la executarea instrucțiunilor

... LA MULTIPROGRAMARE

- Când un job așteaptă terminarea unei instrucțiuni de i/e procesorul poate comuta la un alt job



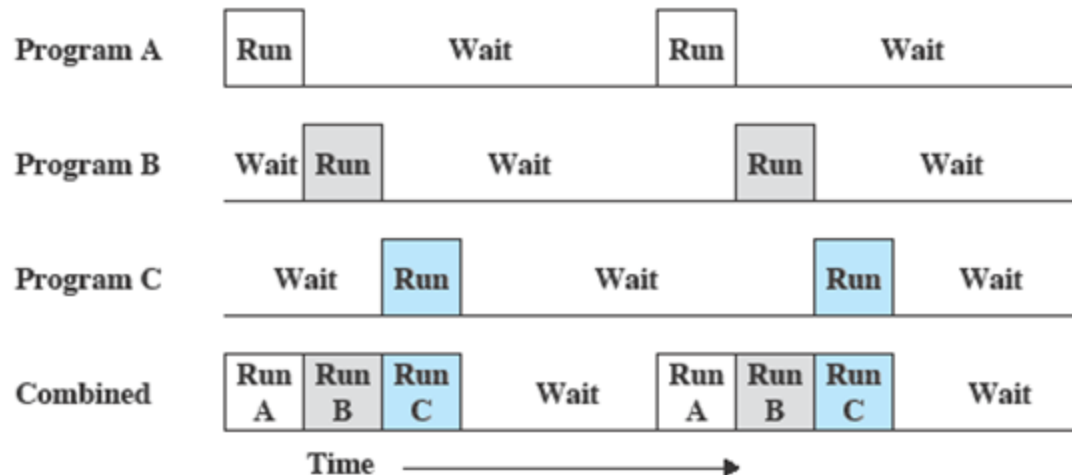
(b) Multiprogramming with two programs

Stallings, pg. 57

MULTIPROGRAMARE

- Memoria trebuie expandată să rețină trei, patru sau mai multe joburi astfel ca procesorul să poată comuta între acestea
- Protejarea memoriei (alt job să nu poată modifica)
- Gestiunea concurenței – corectitudine
 - Sisteme preemptive

Stallings, pg. 57



(c) Multiprogramming with three programs

EVOLUȚIA SO

○ Multiprogramarea

- Eficientizarea utilizării resurselor
 - Organizarea joburilor astfel încât procesorul are tot timpul câte un job de executat
 - Subset din toate joburile este menținut în memorie
 - Selectarea unui job – **planificare** de joburi (job scheduling)
 - Unele joburi stocate pe disc – care așteaptă alocare în memoria internă (job pool)
 - Când se așteaptă după I/E -> procesorul comuta la alt job
- Deficiență: Nu oferă suport pentru interacțiunea cu utilizatorul
 - Timpul de răspuns poate deveni foarte mare
- Soluție: **Time sharing** (partajarea timpului)
 - Numit și **multitasking** – extensie logică a multiprogramării

PROTECȚIE

- Mecanism pentru **izolarea** programelor și utilizatorilor voiti sau involuntar malițioși
 - **SO preemptiv**
 - Previne monopolizarea procesorului
 - Asignează aplicației o resursă și o ia înapoi oferind-o altei aplicații când e nevoie
 - **SO mediator**
 - Se plasează între aplicații și resurse
 - Ține evidența drepturilor (ce poate accesa aplicația)
 - Limitează consumarea resurselor (ex. număr maxim de procese)
 - **Două moduri de execuție pe procesor**
 - Modul neprivilegiat (aplicație) – numit **mod utilizator**
 - Majoritatea software-ului rulează ca procese în mod utilizator
 - Modul privilegiat (supervizor) – numit **mod kernel**
 - Acces nelimitat la toate resursele
 - Kernelul sistemului – crează / șterge procese și furnizează acces la hw
 - Operațiile de protecție se pot efectua doar în mod kernel

EVOLUȚIA SISTEMELOR DE OPERARE

○ Etapa 1

- Hardware-ul este scump, oamenii sunt ieftini
 - Procesare serială
 - Procesare batch
 - Sisteme cu multiprogramare

○ Etapa 2

- Hardware-ul este ieftin, oamenii sunt scumpi
 - Time-sharing
 - Utilizatorii folosesc terminale ieftine și partajează serverele

SISTEME TIME-SHARING – 1970 -

- Pot deservi mai multe joburi interactive
 - Mai mulți utilizatori accesează sistemul
- Procesorul este partajat între mai multe joburi
 - Fiecare primește procesorul pentru o anumită durată de timp
- Multiprogramarea batch vs. Time-sharing
 - Obiectiv principal
 - Multiprogramarea batch
 - Maximizarea utilizării procesorului
 - Time-sharing
 - Minimizarea timpului de răspuns

EVOLUȚIA SO

○ Time sharing (multitasking)

- Oferă suport pentru interacțiunea cu (mai mulți) utilizatori
- Procesorul execută mai multe joburi (concurrent)
 - Comutarea între ele este foarte rapidă
 - Utilizatorul poate interacționa cu fiecare job
 - Utilizatorul are iluzia că doar el este deservit
- Timpul de răspuns – să fie mic (< 1 sec)
- Programul încărcat în memorie și care este în execuție se numește **proces**
- Dacă mai multe procese sunt disponibile pentru rulare
 - Alegerea următorului proces: **planificarea** proceselor (CPU scheduling)
- Dacă procesul nu încapă în memorie
 - **Swapping** – procese mutate temporar din memorie pe disc și înapoi
 - **Memoria virtuală** – permite rularea proceselor care nu sunt complet încărcate în memorie

EVOLUȚIA SISTEMELOR DE OPERARE

○ Etapa 1

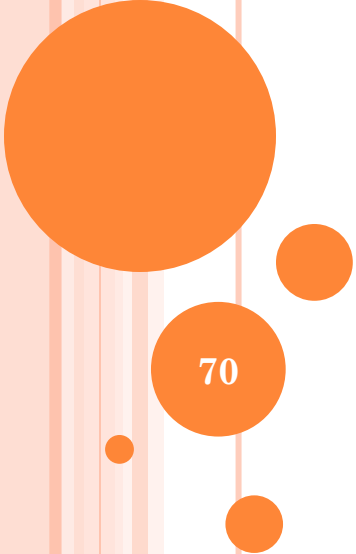
- Hardware-ul este scump, oamenii sunt ieftini
 - Utilizator la consolă
 - Procesare serială
 - Procesare batch
 - Sisteme cu multiprogramare

○ Etapa 2

- Hardware-ul este ieftin, oamenii sunt scumpi
 - Time-sharing
 - Utilizatorii folosesc terminale ieftine și partajează serverele

○ Etapa 3

- Hardware-ul este foarte ieftin, oamenii sunt foarte scumpi
 - Calculatoare personale: un sistem / utilizator
 - Calcul distribuit: mai multe sisteme / utilizator



70

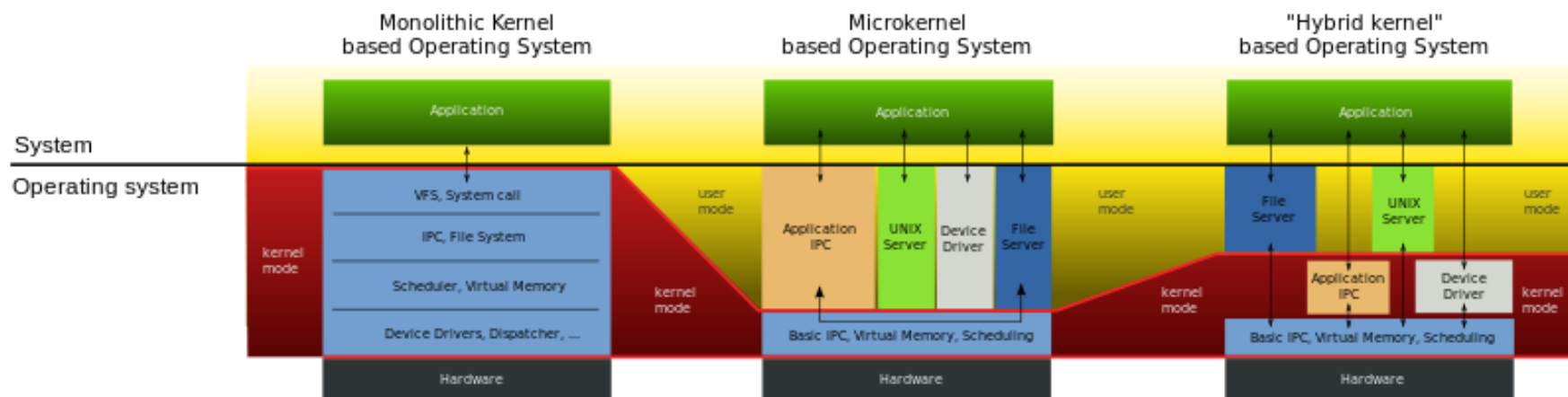
STRUCTURA SO

Monolithic, Microkernel, Hibrid

STRUCTURA SO

- La modul general sistemele de operare constau din doua parti:
 - **spatiul kernel** – care ruleaza in mod privilegiat si
 - **spatiul utilizator** – care ruleaza in mod neprivilegiat
- Separarea este o **necesitate** mai degraba decat o optiune
 - altfel protectia proceselor nu poate fi realizata.
- In functie de **care procese in ce spatiu ruleeaza** putem clasifica sistemele de operare in trei mari arhitecturi
 - Kernel monolitic
 - Microkernel
 - Kernel hibrid sau modular.

https://en.wikipedia.org/wiki/Monolithic_kernel



STRUCTURA SO

○ SO monolitice

- Fără structură – structura e doar logică
- Tot SO rulează în modul kernel – într-un singur spațiu de adrese
- SO este o colecție de proceduri
 - Fiecare este vizibilă fiecăreia (fără ascunderea informației)
 - Oricare poate apela pe oricare alta
- Codul kernel-ului se obține prin compilarea tuturor fișierelor sursă și linkeditarea împreună într-un singur fișier obiect
- Exemple
 - Cele mai multe versiuni UNIX, Linux, MS Windows

STRUCTURA SO

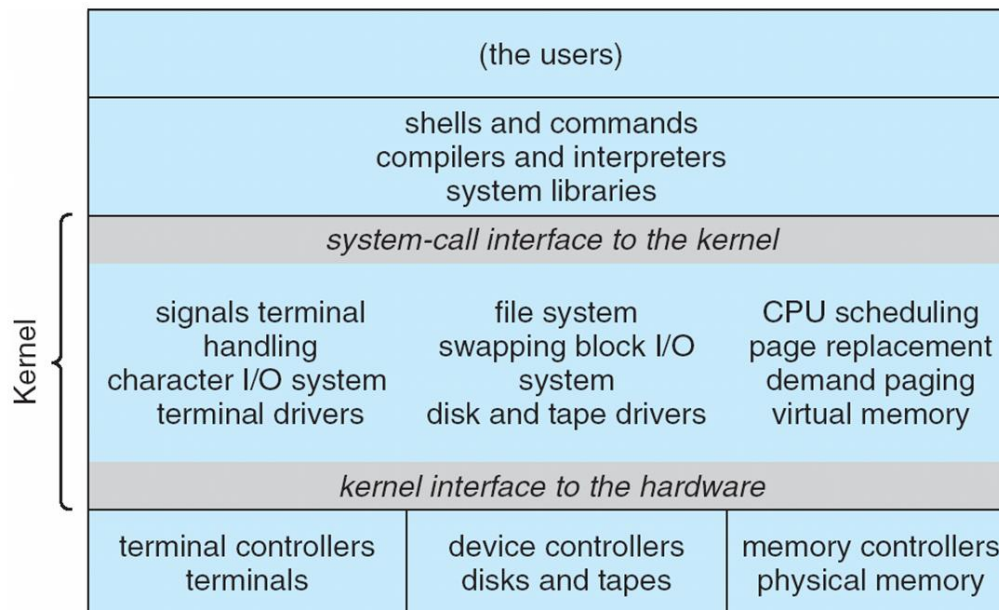
○ SO monolitice (cont.)

○ Avantaj

- Performanță foarte bună – viteză mare
- Dimensiuni relativ reduse pt cod sursă și compilat
- Implementare simplă

○ Dezavantaje

- Poate deveni foarte mare și complex – modificare și extindere mai costisitoare
 - Modificarea implică recompilarea întregului cod din nou
- Dificultate în portarea pe alte platforme
- O eroare la nivelul unei singure proceduri poate duce la eșuarea întregului sistem

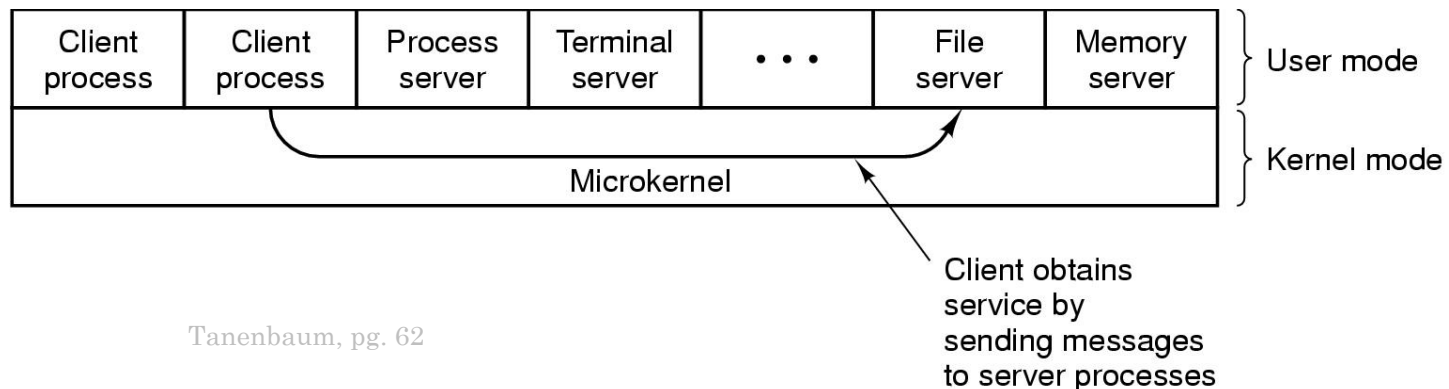


Silberschatz, pg. 71

STRUCTURA SO

○ SO microkernel

- Mutarea multor servicii din spațiul kernel în sp. utilizator
- O parte restrânsă din SO (kernelul) rulează în mod privilegiat
 - Oferă doar mecanisme comunicare între procese și controlul I/O
- Celelalte servicii ale SO (gestiunea memoriei, drivere de dispozitiv, sistemul de fișiere) rulează în mod utilizator
 - Aceste servicii se numesc **servere** – separate, rulează în sp. de adrese diferite
 - Comunicare prin transmitere de mesaje IPC
- Ex. Apple Mac OS X, Minix, L3/L4, etc
- Avantaje
 - Fiabilitate, persistență, modularitate, protecție, portabilitate, adaptabilitate la mediu distribuit
- Dezavantaje – performanță mai mică datorită folosirii IPC între kernel și sp.utilizator
- **preferate acolo unde stabilitatea, siguranța și controlul strict sunt critice**
 - Automotive, aviație, automatizări industriale, sisteme în timp-real

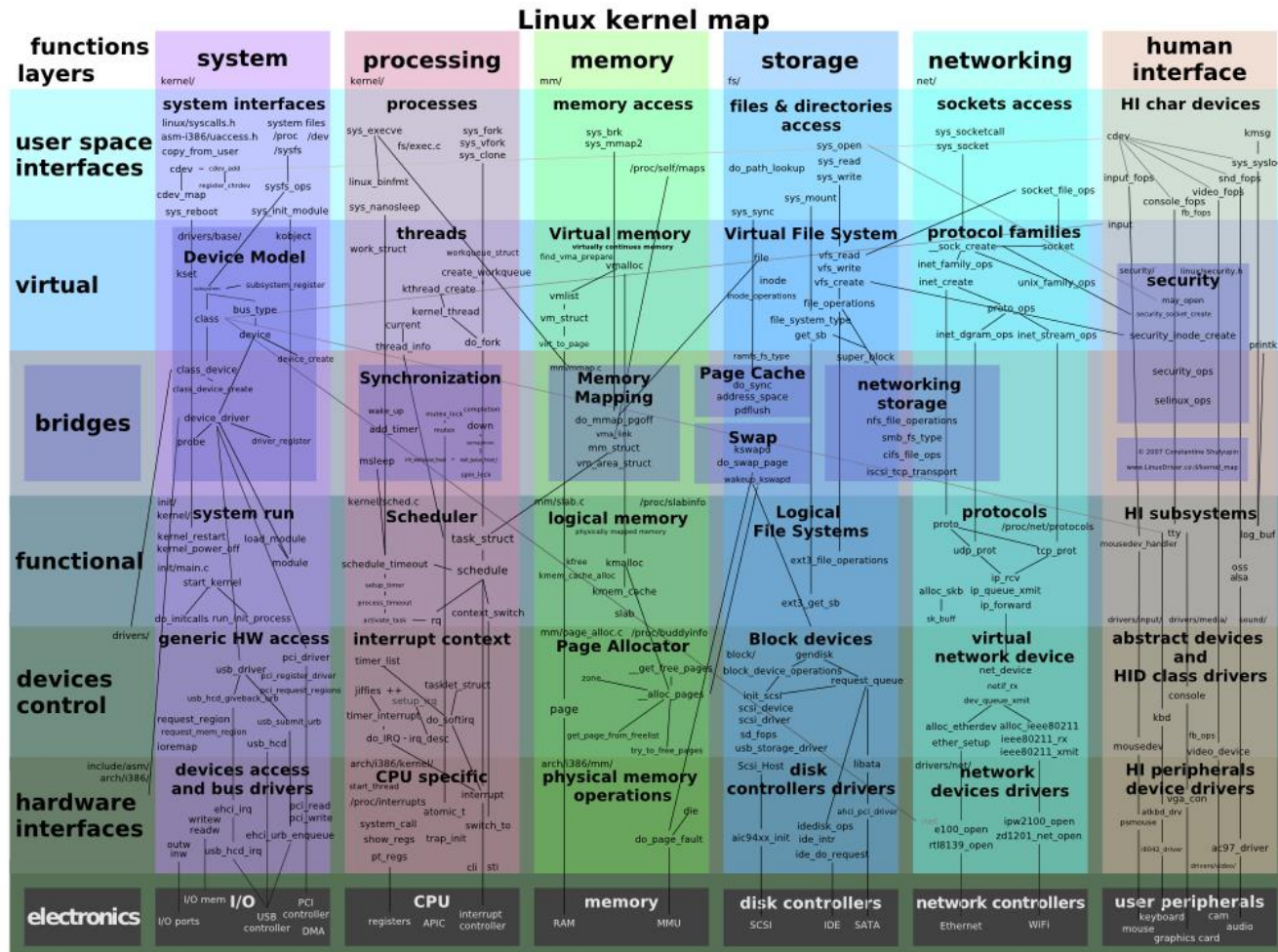


STRUCTURA SO

https://en.wikipedia.org/wiki/Linux_kernel

SO – Linux

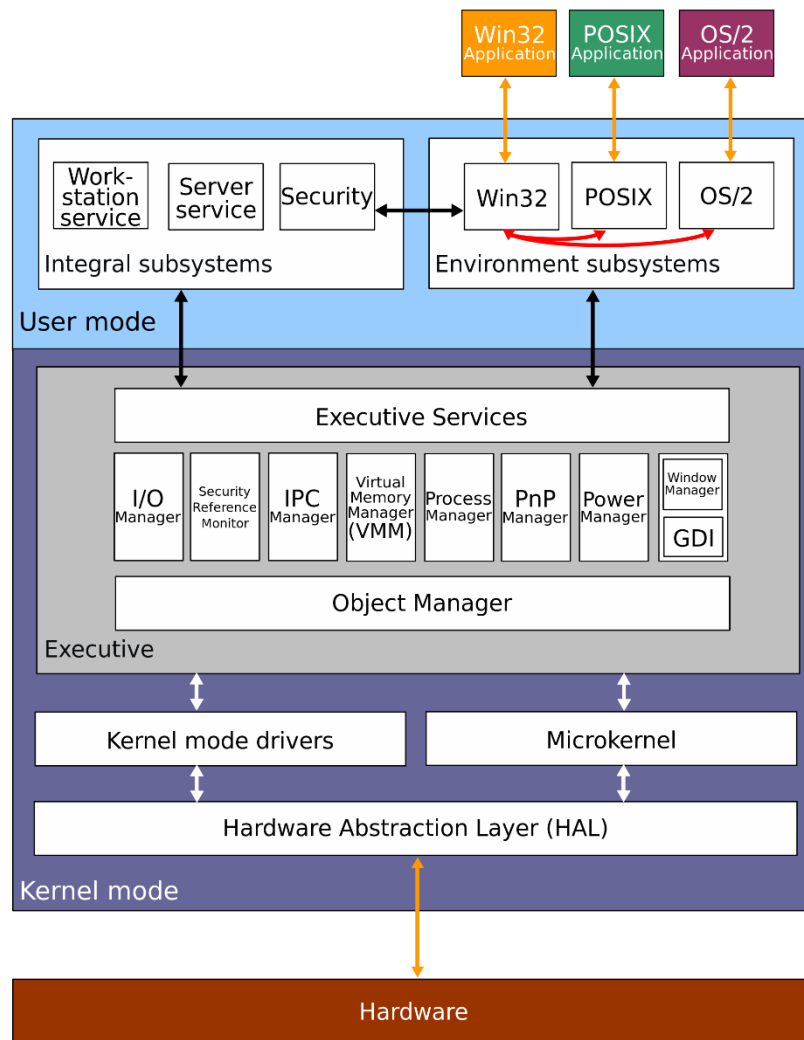
- Structură **monolitică** -> orientare spre hibridă
- Permite adăugarea **modulelor** în timpul rulării
 - Elemente de structură microkernel



STRUCTURA SO

○ SO – Windows NT

- Structură **hibridă modulară**
 - Model client/server pentru implementarea multor funcționalități în spațiul utilizator
 - Ca un microkernel
 - Kernel relativ mare, organizat pe straturi
 - Suport pentru module separate
 - Izolare între user – kernel mode
 - Orientare către performanță și compatibilitate pe sisteme generale
-
- **Hardware → HAL → Kernel → Executive → Drivers → User Mode**



https://en.wikipedia.org/wiki/Hybrid_kernel

